



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

TFG Topics

**Análisis de temas en Trabajos Fin de
Grado**

Documentación Técnica



Presentado por Zeldan Javier Campos Cordero
en Universidad de Burgos — 25 de mayo
de 2026

Tutor: D. Carlos López Nozal

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Metodología y planificación temporal	1
A.3. Análisis de riesgos	4
A.4. Estudio de viabilidad	6
Apéndice B Especificación de Requisitos	9
B.1. Introducción	9
B.2. Objetivos generales	10
B.3. Catálogo de requisitos	11
B.4. Especificación de CUs	12
B.5. Matriz de trazabilidad	19
Apéndice C Especificación de diseño	21
C.1. Introducción	21
C.2. Diseño de datos	23
C.3. Diseño arquitectónico	26
C.4. Diseño procedimental	27
Apéndice D Documentación técnica de programación	33
D.1. Introducción	33

D.2. Arquitectura del Sistema	34
D.3. Estructura de directorios	35
D.4. Manual del programador	36
D.5. Compilación, instalación y ejecución del proyecto	38
D.6. Pruebas del sistema	43
Apéndice E Documentación de usuario	45
E.1. Introducción	45
E.2. Requisitos del sistema	46
E.3. Instalación y despliegue	46
E.4. Manual del usuario de la interfaz	47
Apéndice F Anexo de sostenibilización curricular	59
F.1. Introducción	59
F.2. Sostenibilidad Ambiental	59
F.3. Sostenibilidad Social y Ética	60
F.4. Sostenibilidad Económica	61
F.5. Conclusión	61
Acrónimos	63
Bibliografía	65

Índice de figuras

A.1. Vista del panel de gestión de Sprints en la plataforma Zube . . .	3
A.2. Diagrama de Gantt con la planificación temporal del proyecto . .	3
B.1. Diagrama de CUs del sistema <i>TFG Topics</i>	14
C.1. Diagrama de flujo de datos del sistema	25
C.2. Esquema de componentes basado en Arquitectura Hexagonal . .	27
C.3. Flujo procedimental general del sistema	28
C.4. Diagrama de actividad de la fase de Ingesta	29
C.5. Diagrama de actividad de la fase de Procesamiento	30
C.6. Diagrama de actividad del módulo de Interfaz Visual	31
D.1. Diagrama conceptual de la Arquitectura Hexagonal aplicada en los módulos backend	34
D.2. Diagrama de componentes global del sistema y flujo de datos persistente	35
D.3. Diagrama de interacción y flujo de datos en Apache Parquet entre contenedores Docker	41
D.4. Ejemplo de ejecución del pipeline de pruebas con Pytest	44
E.1. Menu lateral de navegación	48
E.2. Vista Análisis Académico	49
E.3. Vista Distribución Académica	50
E.4. Vista Modo Técnico	51
E.5. Vista Resumen Ejecutivo - Análisis por modelo	52
E.6. Vista Nube de Palabras (Wordcloud) - Exploración de temas . .	53
E.7. Vista Mapa Intertópico	54
E.8. Vista Evolución del score - Hiperparametrización	55
E.9. Vista Mejores hiperparámetros - Hiperparametrización	56

E.10.Vista Comparativa Modelo - Ranking modelos	57
E.11.Vista Comparativa Modelos - Resumen comparativo	58

Índice de tablas

A.1. Cronología de sprints y grado de ejecución del proyecto	5
A.2. Tabla resumen del presupuesto estimado del proyecto	7
B.1. CU-1: Ejecutar pipeline de ingesta.	15
B.2. CU-2: Entrenar modelos de temas.	16
B.3. CU-3: Explorar temas de un modelo.	17
B.4. CU-4: Comparar rendimiento de modelos.	18
B.5. CU-5: Consultar tendencias y metadatos académicos.	19
B.6. Matriz de trazabilidad entre CUs y RFs.	20
C.1. Resumen de decisiones de diseño y sus principales trade-offs. . .	22
F.1. Resumen del impacto de las decisiones técnicas en la sostenibilidad.	60

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Esta sección se detalla la gestión y planificación del **Trabajo de Fin de Grado (TFG)**. Se aborda la metodología de trabajo seguida, la temporalización de las tareas a lo largo de los meses de desarrollo, la gestión de riesgos y, finalmente, un estudio de viabilidad que engloba tanto los aspectos económicos como las consideraciones legales del software desarrollado.

A.2. Metodología y planificación temporal

Para la organización del trabajo, se ha optado por seguir un marco de trabajo ágil adaptado a las necesidades de un proyecto individual académico, basándose en los principios fundamentales de **Scrum**. El desarrollo ha abarcado un periodo total de casi ocho meses, dando comienzo el 16 de octubre de 2025 y finalizando con su presentación en junio de 2026.

El ciclo de vida del desarrollo se estructuró a alto nivel en seis grandes fases consecutivas:

1. **Investigación y Viabilidad:** Análisis del estado del arte y selección tecnológica.
2. **Módulo de Ingesta:** Desarrollo de los adaptadores para extraer datos de GitHub.
3. **Procesamiento NLP y Modelado:** Fase central de experimentación con modelos de temas.

4. **Interfaz Web:** Desarrollo de los cuadros de mando interactivos en Streamlit.
5. **Pruebas y Despliegue:** Refactorización, creación de tests y contene-rización Docker.
6. **Redacción de la Memoria:** Documentación técnica y académica en L^AT_EX.

A nivel operativo, el proyecto se dividió en iteraciones o *Sprints* de entre dos y tres semanas de duración, ajustando este margen de flexibilidad en función de la complejidad del módulo a desarrollar en cada momento. Las reuniones periódicas de seguimiento con los tutores del proyecto ejercieron la doble función de *Sprint Planning* (para priorizar las siguientes tareas) y *Sprint Review* (para evaluar los incrementos de software logrados).

Para el seguimiento de las tareas y el control de versiones, se utilizó la plataforma de gestión ágil **Zube**, la cual se integra nativamente de forma bidireccional con las *issues* de GitHub. Esta herramienta permitió gestionar el *Product Backlog*, asignar prioridades y trazar el progreso visual de cada tarea mediante un tablero Kanban a lo largo de los diferentes *Sprints*, tal y como se ilustra en la Figura A.1.

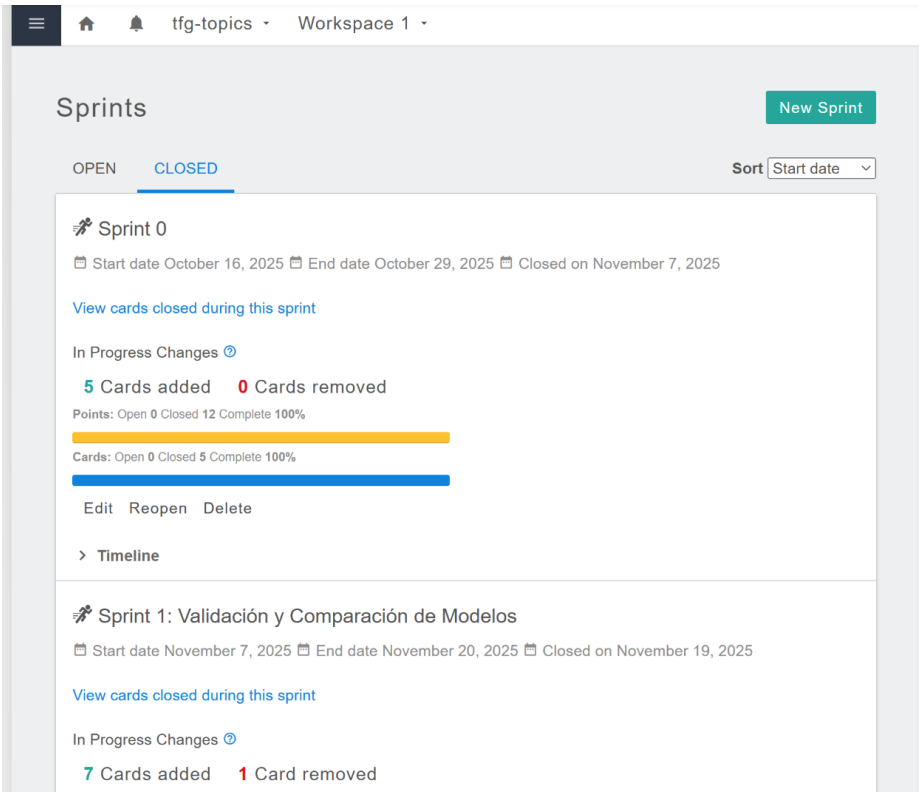


Figura A.1: Vista del panel de gestión de Sprints en la plataforma Zube

La temporalización exacta y la superposición de estas fases se ha modelado en el diagrama de Ganttt adjunto (Figura A.2), que ilustra la línea base del cronograma del proyecto hasta el hito de defensa.

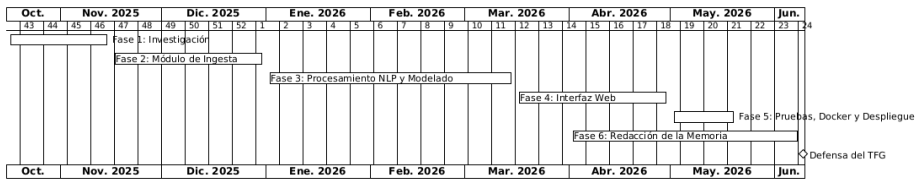


Figura A.2: Diagrama de Gantt con la planificación temporal del proyecto

Evolución real del proyecto por sprints

Además de la planificación inicial, el seguimiento iterativo del desarrollo se gestionó mediante *sprints* cerrados en Zube/GitHub. Esta trazabilidad

permite contrastar el plan previsto con la evolución real del trabajo y evidenciar el grado de cumplimiento.

En total se completaron doce iteraciones (Sprint 0 a Sprint 12), con cierre íntegro de tareas en todos los casos. La evolución del proyecto siguió una secuencia progresiva: arranque y viabilidad inicial, construcción del dataset, consolidación de los módulos de ingesta y procesamiento, mejoras funcionales y visuales en la interfaz, y una fase final orientada a revisión con usuarios, estabilidad e internacionalización. La distribución temporal y el grado de cumplimiento de estas iteraciones se detallan en la Tabla A.1.

El histórico de sprints refleja una planificación adaptativa coherente con la naturaleza experimental del proyecto. Aunque algunas tareas de evaluación y visualización se abordaron en fases tempranas, esta decisión permitió detectar limitaciones de forma anticipada y reorientar con mayor precisión el diseño de ingesta, procesamiento y experiencia de usuario en iteraciones posteriores.

A.3. Análisis de riesgos

Un pilar fundamental en la planificación de cualquier proyecto de ingeniería es la identificación temprana de riesgos y la definición de planes de mitigación. Durante las fases iniciales se identificaron los siguientes riesgos principales:

- **Riesgo 1: Limitaciones de la API de GitHub.** Al depender de una fuente externa para la ingesta de datos, existía el riesgo de alcanzar el límite de peticiones (Rate Limit). *Mitigación:* Se implementó el uso de tokens autenticados, peticiones paginadas y tiempos de retardo controlados (*sleeps*) para garantizar una extracción continua sin bloqueos.
- **Riesgo 2: Altos tiempos de entrenamiento NLP.** El procesamiento masivo de texto y el entrenamiento concurrente de cuatro modelos de temas (LDA, BERTopic, etc.) suponía un cuello de botella computacional. *Mitigación:* Se optó por utilizar archivos Apache Parquet para agilizar las operaciones de I/O en disco y se aprovechó la aceleración por GPU local para la instanciación de los *embeddings* de los Transformers.
- **Riesgo 3: Incompatibilidad de dependencias de Python.** Las librerías de IA suelen presentar conflictos estrictos de versiones. *Mitiga-*

Sprint	Fecha fin	Objetivo principal	Issues	Estado
S0	29/10/2025	Puesta en marcha del proyecto: configuración inicial, investigación de herramientas y arranque de documentación.	5/5	100 %
S1	20/11/2025	Validación y comparación de modelos de temas (LDA, Top2Vec, BERTopic, FASTopic).	6/6	100 %
S2	04/12/2025	Hiperparametrización y primeras visualizaciones de resultados experimentales.	6/6	100 %
S3	17/12/2025	Construcción del dataset base con <i>issues</i> y <i>readmes</i> de repositorios seleccionados.	4/4	100 %
S4	30/12/2025	Extensión de la ingesta a memorias en L ^A T _E X y puesta en marcha del esqueleto de <i>processing</i> .	4/4	100 %
S5	15/01/2026	Implementación del módulo de procesamiento y consolidación del flujo analítico.	7/7	100 %
S6	29/01/2026	Conversión a aplicación desplegable, mejoras de experiencia de usuario y extensión funcional.	3/3	100 %
S7	19/02/2026	Mejora de usabilidad en la página de análisis académico de TFG.	5/5	100 %
S8	05/03/2026	Incorporación del <i>datasource</i> de resúmenes académicos (<i>abstracts</i>).	6/6	100 %
S9	26/03/2026	Mejora visual de la aplicación web y refinamiento de la presentación de resultados.	7/7	100 %
S10	20/04/2026	Revisiones con usuarios y ajustes derivados de pruebas funcionales.	4/4	100 %
S11	04/05/2026	Refinamiento UX, datos demo para validación, estabilización de plataforma e internacionalización.	6/6	100 %
S12	21/05/2026	Usabilidad, Documentación y Despliegue.	6/6	100 %

Tabla A.1: Cronología de sprints y grado de ejecución del proyecto

ción: Se abordó mediante la contención temprana, aislando el entorno de desarrollo en contenedores Docker y especificando versiones exactas en un archivo `requirements.txt`.

A.4. Estudio de viabilidad

Todo proyecto de ingeniería de software requiere de un análisis previo de viabilidad para garantizar que su ejecución sea realista en términos de recursos, coste y adecuación al marco normativo vigente.

Viabilidad económica

Al tratarse de un **TFG** de naturaleza académica, no existe un presupuesto de inversión comercial real. No obstante, es imperativo cuantificar el valor del desarrollo para estimar cuál habría sido su coste total si este proyecto se hubiese encargado dentro de un entorno empresarial. El análisis contempla tres partidas principales:

- **Costes de personal:** La carga lectiva de un **TFG** estándar de 12 créditos ECTS equivale habitualmente a unas 300 horas de dedicación. Asumiendo el rol de un Desarrollador de Software Junior / Ingeniero de Datos con un coste para la empresa estimado de 15 €/hora, el coste salarial ascendería a **4.500 €**.
- **Costes de hardware (Amortización):** El desarrollo de las herramientas y el entrenamiento de los algoritmos de modelado requieren alta capacidad de cómputo. Para ello, se ha utilizado un equipo propio (AMD Ryzen 5 7600X, 32 GB RAM, NVIDIA RTX 4070 SUPER 12GB). Estimando un valor de compra de 1.800 € y una vida útil de 5 años, la amortización correspondiente a los 8 meses del proyecto supone un impacto de **240 €**.
- **Costes de software e infraestructura:** Todos los lenguajes, librerías, y entornos de desarrollo utilizados (Python, Visual Studio Code, Docker, GitHub) disponen de licencias de código abierto o planes gratuitos. En consecuencia, el coste en software es de **0 €**.
- **Gastos generales prorrateados:** Asumiendo un consumo energético y conexión a Internet prorrateado de unos 20 €/mes durante 8 meses de trabajo en oficina u hogar, se estiman **160 €**.

Concepto	Coste estimado
Costes de personal (300h)	4.500 €
Amortización de hardware	240 €
Licencias de software	0 €
Gastos generales	160 €
Presupuesto Total	4.900 €

Tabla A.2: Tabla resumen del presupuesto estimado del proyecto

A continuación, en la Tabla A.2, se muestra de forma resumida el desglose presupuestario del proyecto.

Agregando todas las partidas, el presupuesto total de desarrollo de esta herramienta rondaría los **4.900 €**. Teniendo en cuenta la nula barrera de entrada a nivel de licencias y herramientas, se concluye que el proyecto es totalmente **económicamente viable**.

Viabilidad legal

Desde la perspectiva jurídica, el desarrollo, almacenamiento y futura distribución de esta aplicación en código abierto plantea dos retos principales que han sido solventados:

- **Protección de datos personales:** Dado que el sistema requiere analizar metadatos e incidencias de repositorios que actúan como **TFG**, existía el riesgo de vulnerar la privacidad de autores y tutores. Para solucionarlo, el módulo de extracción se ha configurado para **anonimizar intencionalmente** cualquier nombre o dato de contacto recopilado. Además, la información es extraída de forma no invasiva **exclusivamente desde repositorios públicos** de GitHub, acatando escrupulosamente los Términos de Servicio de su **API** para usos académicos.
- **Licenciamiento del software:** El código desarrollado hace uso de librerías de terceros (tales como *Pandas* o *Streamlit*) amparadas bajo licencias permisivas (BSD o Apache 2.0). Para asegurar la mayor interoperabilidad y evitar bloqueos en usos posteriores por otros investigadores, el código fuente completo de este proyecto será liberado en GitHub bajo la **Licencia MIT**. Esta licencia, al ser altamente

permisiva, garantiza que cualquier usuario pueda modificar y distribuir el código base, requiriendo únicamente preservar el aviso de derechos de autor original, lo que consolida por completo su **viabilidad legal**.

Apéndice B

Especificación de Requisitos

B.1. Introducción

En esta sección se documenta el proceso de ingeniería de requisitos llevado a cabo durante las fases iniciales del proyecto. A partir del análisis del problema y de las necesidades del dominio analítico, se han establecido los objetivos principales del sistema y se ha redactado un catálogo formal detallando las funcionalidades esperadas (**Requisitos Funcionales (RFs)**) y las restricciones técnicas impuestas a la arquitectura (**Requisitos No Funcionales (RNFs)**).

Finalmente, se modela el comportamiento del sistema desde la perspectiva de sus usuarios mediante la especificación de **Casos de Uso (CUs)**.

Términos del Dominio

Para facilitar la comprensión del catálogo de requisitos y de los flujos de interacción del sistema, se definen a continuación los conceptos clave relativos al dominio de datos analizado:

Trabajo de Fin de Grado (TFG): Proyecto académico y práctico obligatorio que los estudiantes de último curso de Ingeniería Informática desarrollan y defienden para demostrar las competencias y conocimientos adquiridos a lo largo de su titulación.

Readme (Fichero README): Archivo de documentación (usualmente en formato Markdown o texto plano) ubicado en la raíz del repositorio de GitHub que proporciona información introductoria del proyecto,

instrucciones de configuración, despliegue y uso de la aplicación desarrollada.

Abstract (Resumen académico): Síntesis concisa del trabajo (habitualmente redactada tanto en español como en inglés) que resume de forma compacta los objetivos, la metodología empleada y las conclusiones principales del **TFG**.

Issue (Incidencia/Tarea): Herramienta integrada en el repositorio de GitHub para registrar, planificar y realizar el seguimiento de tareas pendientes, errores de programación, debates de diseño o hitos del desarrollo. En este contexto, representan el histórico de actividad y comunicación del proyecto.

Contenido / Memoria del TFG: Texto íntegro contenido en los ficheros fuente en $\text{\LaTeX}$ redactados por el alumno, que constituye el documento analítico y descriptivo formal de su trabajo.

B.2. Objetivos generales

El principal propósito de *TFG Topics* es proporcionar una herramienta que facilite el análisis temático de los **TFG** del grado de Ingeniería Informática de la Universidad de Burgos disponibles en repositorios públicos de GitHub. Para lograrlo, se han definido los siguientes objetivos generales:

- **Extracción de datos:** Extraer el contenido de las memorias de **TFG**, *readmes*, *issues* y *abstracts* de repositorios públicos de GitHub.
- **Implementar procesamiento NLP:** Normalizar los textos extraídos y prepararlos para el modelado temático.
- **Entrenar y evaluar algoritmos de modelado de temas:** Entrenar modelos como **LDA**, BERTopic, Top2Vec y FASTopic para la extracción de temas.
- **Desarrollar una interfaz web:** Permitir a los investigadores explorar de forma interactiva la distribución de temas y comparar empíricamente el rendimiento de cada modelo.

B.3. Catálogo de requisitos

El catálogo de requisitos desglosa los objetivos en unidades atómicas y verificables, dividiéndolas según su naturaleza.

Requisitos Funcionales (RFs)

Definen las funcionalidades y acciones concretas que el sistema debe ser capaz de ejecutar.

- **RF-1. Extracción de datos:** El sistema debe descargar la información de repositorios a partir de un listado de orígenes parametrizado, comunicándose con la **API** de GitHub.
- **RF-2. Preprocesamiento de texto:** El sistema debe aplicar técnicas de procesamiento de lenguaje natural (tokenización, lematización, borrado de *stopwords*) para normalizar el texto.
- **RF-3. Entrenamiento de modelos:** El sistema debe ser capaz de instanciar y entrenar múltiples algoritmos de modelado de temas sobre el texto preprocesado.
- **RF-4. Optimización de hiperparámetros:** El sistema debe integrar mecanismos para evaluar y afinar los hiperparámetros de los modelos en busca de la máxima coherencia semántica.
- **RF-5. Cálculo de métricas:** El sistema debe generar métricas estandarizadas para evaluar la coherencia y diversidad de los temas.
- **RF-6. Visualización de temas:** La interfaz de usuario debe mostrar gráficas interactivas que representen la importancia de las palabras y de los temas.
- **RF-7. Comparativa de modelos:** La interfaz debe disponer de un panel que cruce y ordene el rendimiento métrico de los diferentes algoritmos.
- **RF-8. Análisis exploratorio académico:** La interfaz de usuario debe permitir filtrar los proyectos (por tutor, rango de años y calificaciones) y visualizar gráficos sobre la distribución y evolución de temáticas por curso académico.

Requisitos No Funcionales (RNFs)

Restricciones de diseño, rendimiento y arquitectura que garantizan la calidad y escalabilidad del producto final.

- **RNF-1. Persistencia optimizada:** Dado el alto volumen de datos textuales, el almacenamiento local entre los módulos del sistema debe realizarse exclusivamente en formato Apache Parquet.
- **RNF-2. Desacoplamiento estructural:** La lógica de negocio del sistema debe aislarse de los detalles técnicos y librerías externas siguiendo el patrón de Arquitectura Hexagonal.
- **RNF-3. Contenerización:** Para evitar conflictos de dependencias, la ejecución de los diferentes módulos debe estar soportada por contenedores Docker independientes.
- **RNF-4. Tolerancia a fallos de red:** El módulo de ingesta debe gestionar de forma silenciosa la posible pérdida de conexión o la caída de la **API** remota mediante reintentos y persistencia parcial.

B.4. Especificación de CUs

La especificación de **Casos de Uso (CUs)** modela las interacciones entre los usuarios (o sistemas externos) y la aplicación para lograr objetivos específicos.

Catálogo de Actores

Para acotar el alcance de las interacciones, se definen a continuación los roles o sistemas externos que participan en el sistema analítico:

Administrador / Desarrollador (Humano, Primario Backend): Perfil con conocimientos técnicos encargado de la configuración del sistema, la puesta en marcha de los entornos y la ejecución desatendida del pipeline de datos en sus etapas de ingesta y procesamiento desde la CLI.

Usuario Académico (Humano, Primario Frontend): Docente o estudiante que busca respuestas funcionales en el corpus analizado. Su objetivo es identificar qué tecnologías se investigan más, cómo evolucionan las temáticas a lo largo de los cursos y qué tendencias existen

en los TFGS de Ingeniería Informática, todo ello a través de filtros y distribuciones temporales sencillas y sin necesidad de conocer en detalle el modelado de temas.

Usuario Técnico / Investigador IA (Humano, Primario Frontend):

Experto en NLP que evalúa la calidad científica y estadística de los resultados. Utiliza la interfaz web técnica para analizar proyecciones, comprobar curvas de optimización de hiperparámetros y comparar cuantitativamente las métricas de coherencia entre los algoritmos.

API de GitHub (Sistema Externo, Secundario): Interfaz de programación provista por GitHub que actúa como proveedor de datos externo. Es consumida por el pipeline para descargar de forma automatizada el contenido de las memorias de TFG, *readmes*, *issues* y *abstracts* del repositorio semilla.

Diagrama de Casos de Uso

El panorama general de interacciones queda plasmado en el diagrama de CUs adjunto (Figura B.1).

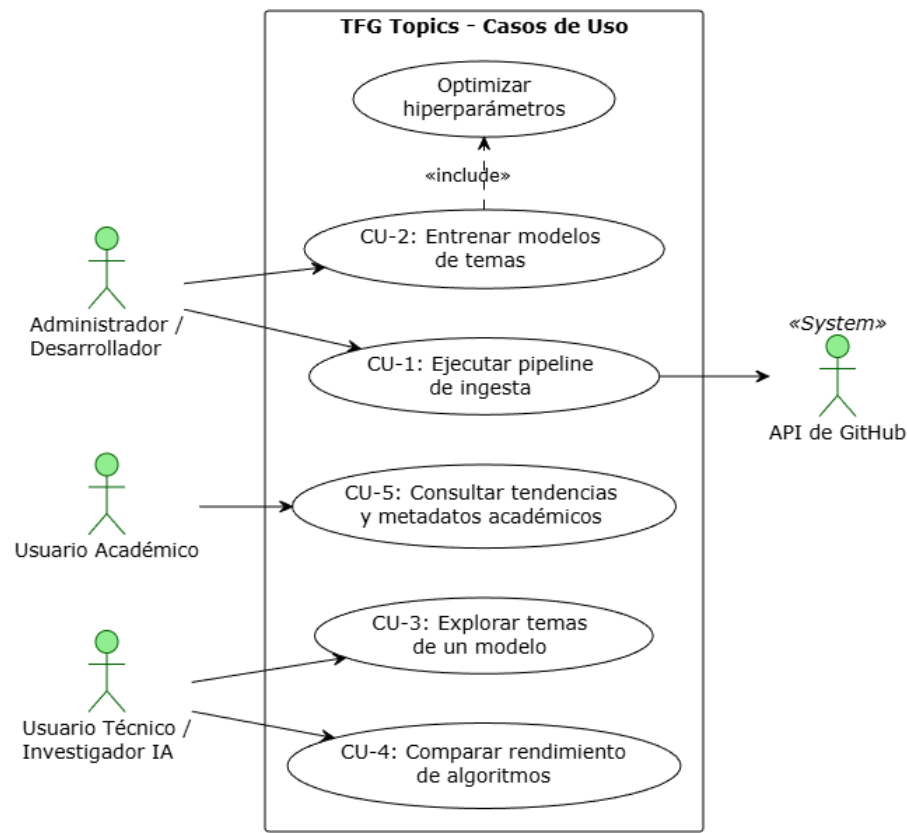


Figura B.1: Diagrama de CUs del sistema *TFG Topics*

Fichas Tabulares de Casos de Uso

A continuación, se documentan de forma tabular los flujos detallados asociados a cada uno de estos CUs.

CU-1	Ejecutar pipeline de ingesta
Versión	1.0
Autor	Alumno
Actores	Principal: Administrador / Desarrollador. Secundario: API de GitHub (Sistema externo).
Requisitos asociados	RF-1, RNF-1, RNF-4
Descripción	El administrador inicia la recolección de los datos desde la nube.
Precondición	Disponer de conexión a Internet, de un archivo semilla de repositorios y de un token de GitHub válido.
Acciones	1. El administrador lanza la orden de ejecución. 2. El sistema lee el fichero semilla. 3. El sistema consulta la <i>API</i> y descarga el contenido de las memorias de <i>TFG</i>, <i>readmes</i>, <i>issues</i> y <i>abstracts</i>. 4. El sistema formatea los datos y los persiste.
Postcondición	Se han generado los archivos con los datos extraídos.
Excepciones	Fallo de red, token expirado o repositorios eliminados (se registran en los <i>logs</i> sin detener la ejecución).
Importancia	Alta

Tabla B.1: CU-1: Ejecutar pipeline de ingesta.

CU-2	Entrenar modelos de temas
Versión	1.0
Autor	Alumno
Actores	Principal: Administrador / Desarrollador.
Requisitos asociados	RF-2, RF-3, RF-4, RF-5
Descripción	El administrador ejecuta el proceso de análisis de los datos extraídos.
Precondición	Deben existir archivos de ingesta generados por el CU-1.
Acciones	1. El administrador lanza el módulo de <i>processing</i>. 2. El sistema carga en memoria los textos crudos y aplica los filtros de limpieza NLP. 3. El sistema entrena iterativamente los diferentes algoritmos. 4. El sistema optimiza automáticamente los hiperparámetros de los modelos. 5. Se calculan las métricas para la vista final.
Postcondición	Se han generado los ficheros con los resultados analíticos correspondiente a cada modelo.
Excepciones	Insuficiente memoria RAM para cargar un corpus completo (se aborta el hilo).
Importancia	Alta

Tabla B.2: CU-2: Entrenar modelos de temas.

CU-3	Explorar temas de un modelo
Versión	1.0
Autor	Alumno
Actores	Principal: Usuario Técnico / Investigador IA.
Requisitos asociados	RF-6
Descripción	El usuario técnico visualiza las temáticas descubiertas por un algoritmo en particular de forma interactiva.
Precondición	El sistema web está arrancado y los modelos han sido entrenados. Deben existir archivos de ingesta generados por el CU-1. Deben existir archivos generados por el CU-2.
Acciones	1. El usuario accede a la pestaña de análisis de modelo en el <i>frontend</i>. 2. Selecciona mediante un desplegable la fuente de texto y el modelo objetivo. 3. El sistema carga bajo demanda la información del modelo. 4. El usuario interactúa con los gráficos.
Postcondición	El usuario adquiere conclusiones sobre el corpus.
Excepciones	Datos no disponibles si el modelo no fue entrenado (el <i>frontend</i> muestra advertencia).
Importancia	Alta

Tabla B.3: CU-3: Explorar temas de un modelo.

CU-4	Comparar rendimiento de modelos
Versión	1.0
Autor	Alumno
Actores	Principal: Usuario Técnico / Investigador IA.
Requisitos asociados	RF-7
Descripción	El usuario técnico coteja las métricas de distintos algoritmos para evaluar cuál se comporta mejor ante el conjunto de datos.
Precondición	Al menos dos modelos diferentes deben haber sido entrenados para la misma fuente de datos.
Acciones	1. El usuario accede a la pestaña de comparativa. 2. El sistema agrega la puntuación de coherencia de todos los modelos disponibles. 3. Se renderiza una tabla de clasificación o <i>ranking</i> que consolida el rendimiento. 4. El usuario puede visualizar gráficos comparativos.
Postcondición	Ninguna alteración del estado del sistema.
Excepciones	Ninguna.
Importancia	Media

Tabla B.4: CU-4: Comparar rendimiento de modelos.

CU-5	Consultar tendencias y metadatos académicos
Versión	1.0
Autor	Alumno
Actores	Principal: Usuario Académico.
Requisitos asociados	RF-8
Descripción	El usuario académico filtra los proyectos y analiza visualmente la evolución de temas y tecnologías por curso.
Precondición	La interfaz web está iniciada y dispone de los datos de los proyectos cargados.
Acciones	1. El usuario accede a la pestaña de análisis académico en el <i>frontend</i>. 2. Aplica filtros por rango de años, notas o nombre del tutor si lo desea. 3. El sistema actualiza el listado y muestra las fichas de los proyectos correspondientes. 4. El usuario visualiza la distribución de temas y evolución de tecnologías en los gráficos interactivos de la sección.
Postcondición	Se presenta la información filtrada y los gráficos de evolución correspondientes.
Excepciones	Ninguna.
Importancia	Alta

Tabla B.5: CU-5: Consultar tendencias y metadatos académicos.

B.5. Matriz de trazabilidad

Para asegurar que todos los **RFs** identificados están cubiertos por las interacciones de los usuarios con el sistema, se presenta a continuación la matriz de trazabilidad entre los **CUs** y los **RFs**.

	RF-1	RF-2	RF-3	RF-4	RF-5	RF-6	RF-7	RF-8
CU-1	X							
CU-2		X	X	X	X			
CU-3						X		
CU-4							X	
CU-5								X

Tabla B.6: Matriz de trazabilidad entre CUs y RFs.

Apéndice C

Especificación de diseño

C.1. Introducción

Esta sección detalla las decisiones técnicas y estructurales adoptadas para transformar los requisitos del sistema en una solución de software funcional y mantenible. El objetivo principal del diseño ha sido establecer una base sólida que soporte las distintas fases del proyecto, desde la recopilación de datos en la plataforma GitHub hasta la aplicación de los algoritmos de modelado de temas y su posterior visualización.

Para lograrlo, el diseño se ha abordado desde tres perspectivas complementarias: la estructuración y persistencia de los datos, la arquitectura de los componentes software y, por último, la definición de los procesos y flujos de ejecución. A la hora de tomar estas decisiones de diseño, se ha priorizado la modularidad del código, el rendimiento en el manejo de los volúmenes de texto extraídos y la facilidad para poder incorporar nuevos modelos en el futuro sin alterar la base existente.

Criterios de decisión y trade-offs

Antes de definir los artefactos concretos del sistema, se establecieron varios criterios de diseño para guiar las decisiones técnicas: mantenibilidad del código, reproducibilidad experimental, rendimiento sobre colecciones textuales grandes y facilidad de evolución del proyecto por parte de futuros estudiantes.

A partir de estos criterios, se adoptaron las siguientes decisiones:

- **Arquitectura Hexagonal en los módulos backend:** Se priorizó el desacoplamiento entre la lógica de negocio y los detalles de infraestructura para reducir el coste de mantenimiento y facilitar las pruebas unitarias [15]. Como contrapartida, se asumió una mayor complejidad inicial en la organización del código y en la definición de puertos/adaptadores.
- **Parquet como formato estándar de intercambio:** Se descartaron formatos como **JSON** o **CSV** para el flujo interno, debido a su menor eficiencia en lectura y mayor consumo de almacenamiento en escenarios de volumen. Parquet permitió mejorar tiempos de acceso y compresión, a cambio de introducir una dependencia más fuerte del ecosistema analítico de Python [13].
- **Pipeline modular por etapas (ingesta, procesamiento y visualización):** Se separaron las fases para poder reejecutar únicamente el tramo necesario según el tipo de cambio realizado (datos, modelos o interfaz). Esta modularidad incrementa la trazabilidad, aunque obliga a definir contratos de datos explícitos entre módulos.
- **Estrategia de ejecución multi-entorno (MODE=release, MODE=docker, MODE=local):** Se buscó equilibrar accesibilidad para usuarios no técnicos y flexibilidad para desarrollo. El coste asociado es mantener y documentar correctamente varios flujos de ejecución.

Decisión	Beneficio principal	Coste asumido
Arquitectura Hexagonal	Desacoplamiento y testabilidad	Mayor complejidad inicial de diseño
Parquet como formato común	Mejor rendimiento y compresión	Dependencia del ecosistema analítico
Pipeline modular por etapas	Reejecución parcial y trazabilidad	Necesidad de contratos de datos explícitos
Multi-entorno de ejecución	Flexibilidad para usuario y desarrollo	Mayor esfuerzo de documentación y validación

Tabla C.1: Resumen de decisiones de diseño y sus principales trade-offs.

Estas decisiones se detallan y materializan en las secciones siguientes de diseño de datos, diseño arquitectónico y diseño procedimental.

C.2. Diseño de datos

El componente analítico de este trabajo requiere manipular, transformar y almacenar una cantidad considerable de texto en crudo y metadatos provenientes de repositorios. Durante las pruebas iniciales de concepto se plantearon formatos de persistencia tradicionales como **JSON** o archivos **CSV**; sin embargo, acabaron siendo descartados.

En su lugar, se ha optado por estandarizar el uso del formato columnar **Apache Parquet** para el almacenamiento local y el paso de mensajes entre todos los módulos del proyecto. Apoyándonos en las librerías **Pandas** y **PyArrow** [13], este formato nos aporta una compresión muy superior y tiempos de acceso prácticamente instantáneos, algo fundamental al procesar los muchos archivos provenientes de las distintas fuentes como *Readmes*, *Issues* y *Abstracts* y demás documentos de texto.

Los datos se estructuran de forma progresiva en diferentes grupos de almacenamiento, comenzando por las fuentes iniciales y desembocando en los resultados del modelado:

1. Datos de Entrada (data/): El flujo arranca con el archivo maestro `tfg_list.csv`. Este fichero actúa como la *semilla* del sistema, albergando el catálogo inicial de repositorios que el módulo de ingesta debe descargar. Internamente, el **CSV** define una estructura tabular con metadatos clave para cada **TFG**: el título del proyecto (`title`), las siglas de los tutores involucrados (`tutors`), la cantidad de alumnos (`students`), las fechas de asignación y presentación (`assignment_date`, `presentation_date`), la calificación obtenida (`grade`) y, lo más importante, el enlace directo (`repository_url`), que sirve como punto de entrada a la **API** de GitHub para comenzar la extracción.

2. Ficheros de Ingesta (data/ingestion/): Contienen la información textual y descriptiva recopilada desde GitHub a partir del listado anterior. Todos los registros comparten como nexo común el identificador del repositorio.

- `metadata.parquet`: Almacena los metadatos principales de los repositorios identificados como potenciales Trabajos de Fin de Grado (nombre, enlaces, fechas, número de estrellas, etc.).

- `issues.parquet`: Contiene el texto extraído de las incidencias o debates abiertos en cada repositorio.
- `readmes.parquet`: Contiene el texto extraído de los ficheros *Readme* principales.
- `thesis.parquet`: Almacena el contenido textual extraído de las memorias de los proyectos al completo.
- `abstracts.parquet`: Guarda únicamente los resúmenes de dichas memorias, sirviendo como corpus alternativo y más directo para el análisis.

3. Ficheros de Procesamiento (`data/processing/`): Para cada algoritmo utilizado (`LDA` [17], `BERTopic` [10], `Top2Vec` [16], `FASTopic` [2]) y para cada fuente de texto analizada, el sistema genera dinámicamente un conjunto de archivos Parquet con los resultados analíticos y del entrenamiento:

- `*_topics.parquet`: Recoge la lista de temas descubiertos y los pesos de las palabras clave asociadas.
- `*_document_topics.parquet`: Guarda la distribución probabilística y la asignación principal de temas para cada documento analizado.
- `*_metrics.parquet`: Contiene las métricas de evaluación del rendimiento (por ejemplo, medidas de coherencia, diversidad, etc.).
- `*_model_summary.parquet`: Ofrece un resumen estructural de los parámetros básicos del modelo entrenado.
- `*_hyperparameter_trials.parquet`: Registra el historial completo de todas las iteraciones de prueba durante el *tuning* o ajuste de hiperparámetros.
- `*_best_hyperparameters.parquet`: Almacena de forma aislada la mejor configuración de hiperparámetros hallada para ese modelo en concreto.
- `*_params.parquet`: Contiene el diccionario general de parámetros bajo los que ha sido ejecutado finalmente el modelo.
- `*_topic_coordinates.parquet`: Almacena las coordenadas dimensionales (normalmente proyecciones 2D) precalculadas, vitales para poder renderizar mapas de distancia de temas de manera instantánea en el *frontend*.

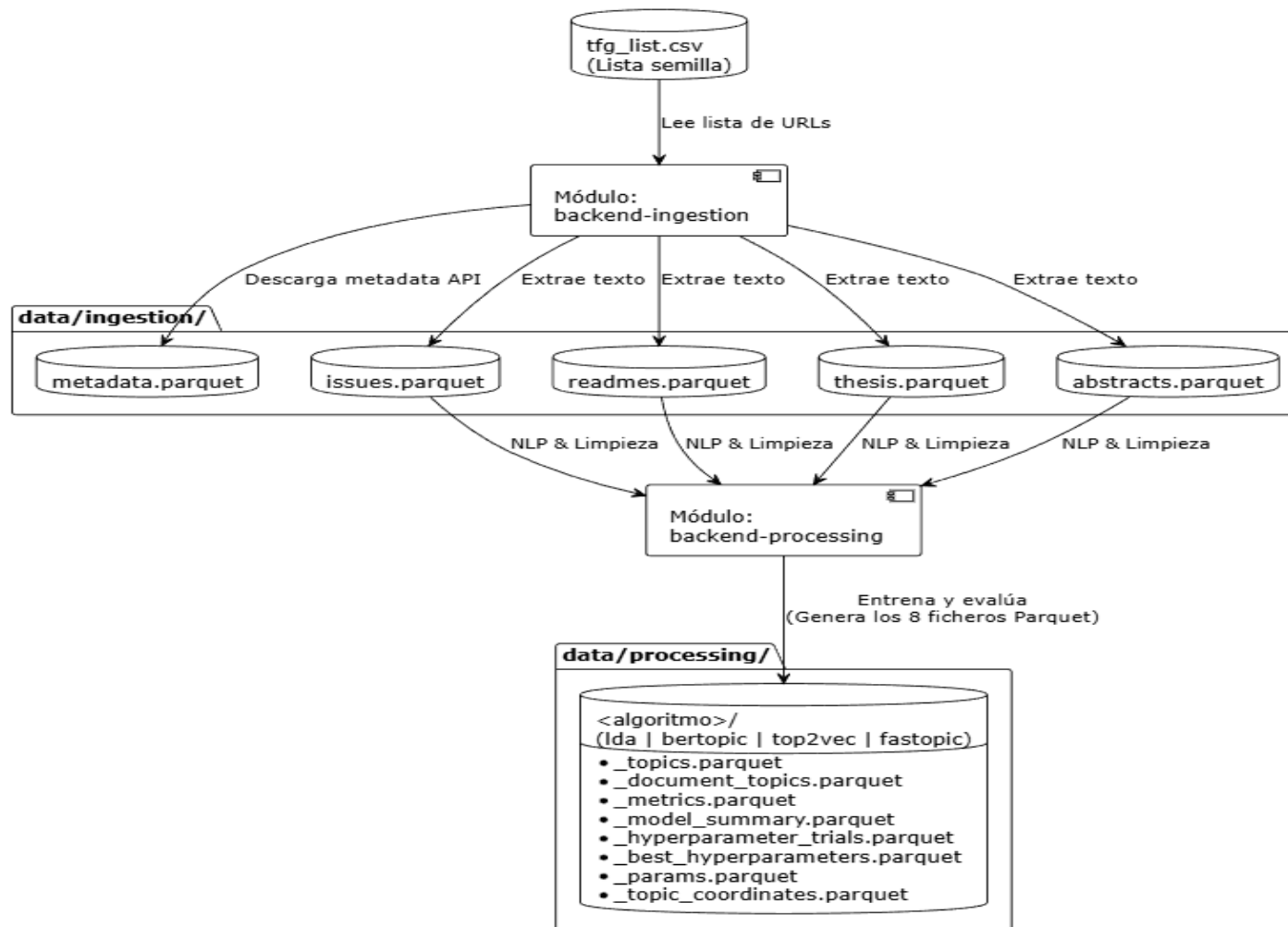


Figura C.1: Diagrama de flujo de datos del sistema

C.3. Diseño arquitectónico

Teniendo en cuenta que el sistema requiere orquestar llamadas a servicios externos, rutinas de **NLP** pesadas y varias librerías de modelado complejas, plantear un diseño tradicional en capas monolítico habría resultado en un código difícil de testear y de mantener. Por esta razón, el núcleo principal del *backend* (los módulos **backend-ingestion** y **backend-processing**) se ha modelado siguiendo estrictamente los principios de la **Arquitectura Hexagonal** [15] (conocida también como patrón de Puertos y Adaptadores).

Este paradigma arquitectónico nos ha permitido aislar por completo la lógica de negocio del proyecto de los detalles puramente técnicos de la infraestructura. La separación de responsabilidades se ha organizado en los siguientes bloques:

1. **Dominio (*Domain*):** Es el corazón de la aplicación. Aquí se han definido las entidades puras y los *puertos* (interfaces declaradas mediante clases abstractas `abc.ABC` en Python). Esta capa es agnóstica; no incluye importaciones de librerías externas ni sabe de la existencia de Parquet o GitHub.
2. **Aplicación (*Application*):** Contiene los *CUs*. Estos módulos son los responsables de orquestar el flujo de la información utilizando exclusivamente las reglas del dominio. Una regla estricta que se ha aplicado en el desarrollo es la prohibición de realizar operaciones de entrada/salida directas en esta capa.
3. **Infraestructura (*Infrastructure*):** En esta capa más externa residen los *adaptadores*, que proporcionan las implementaciones reales para los puertos del dominio. Es aquí donde se encapsulan las llamadas a la **API** de GitHub, las lecturas del sistema de ficheros y las invocaciones a los modelos específicos como **LDA**, BERTopic, Top2Vec o FASTopic.

En cuanto a la interfaz de usuario (**frontend-web**), desarrollada íntegramente con **Streamlit**, su diseño arquitectónico es intencionadamente minimalista. Actúa simplemente como un cliente de sólo lectura que consume los archivos Parquet ya generados por el *backend*, delegando toda la complejidad computacional a los módulos de procesamiento subyacentes [12].

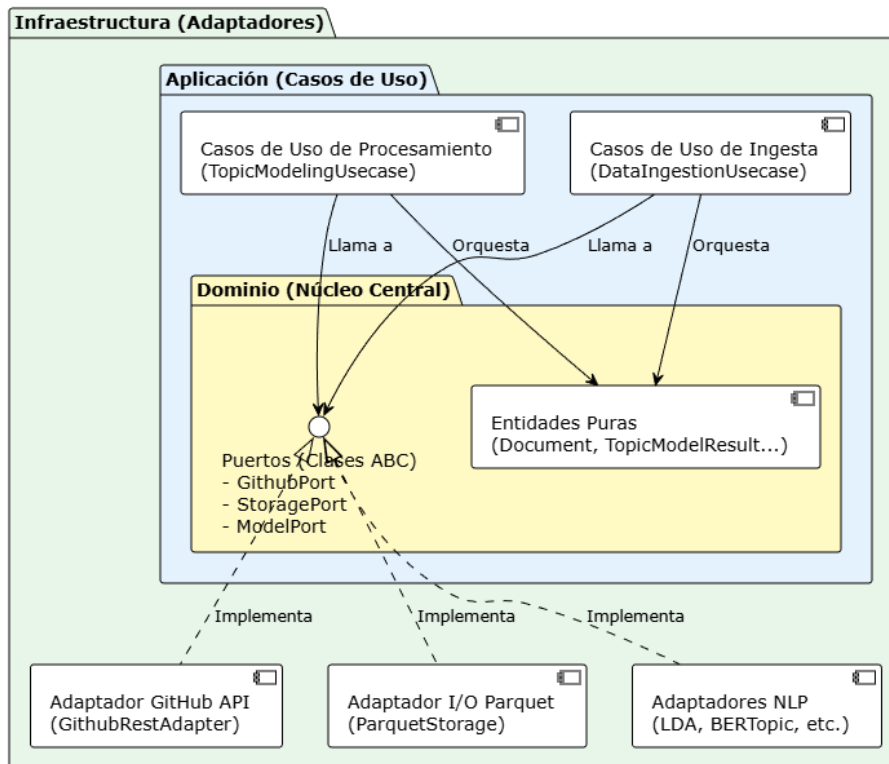


Figura C.2: Esquema de componentes basado en Arquitectura Hexagonal

C.4. Diseño procedimental

Por último, el diseño procedimental especifica la secuencia lógica y el flujo de los algoritmos que permiten transformar los datos brutos en visualizaciones de temas. A nivel global, el ciclo de vida de la información se divide en un *pipeline* de tres grandes fases automatizadas (manejadas a través de *Makefile* y *Docker Compose* [14]):

1. **Ingesta de datos (*Ingestion*):** El flujo se inicia lanzando consultas parametrizadas contra GitHub para buscar repositorios de TFG. El algoritmo itera sobre los resultados de la búsqueda, descargando para cada repositorio sus datos básicos, el archivo léame y las incidencias abiertas. Se aplican filtros en tiempo de ejecución para descartar repositorios con información insuficiente, volcando el resultado válido en los primeros ficheros Parquet.

2. **Procesamiento y Modelado (*Processing*)**: Es el procedimiento central del proyecto. En primer lugar, se ejecuta un algoritmo de pre-procesado de texto que normaliza el contenido (tokenización, reducción a lemas, borrado de palabras vacías y limpieza de código Markdown). Tras esto, el flujo pasa a la fase de entrenamiento, donde el sistema puede instanciar de forma genérica cualquiera de los modelos integrados (**LDA**, BERTopic, etc.). El sistema está programado para permitir el ajuste de hiperparámetros y generar, como salida final, la distribución probabilística de los temas encontrados.
3. **Visualización interactiva (*Frontend*)**: El procedimiento final es la exposición de los datos. A medida que el usuario navega por la interfaz de Streamlit, el sistema lee bajo demanda las porciones necesarias de los ficheros Parquet y calcula en caliente las gráficas y métricas comparativas que se solicitan en la pantalla.

A nivel global, la secuencia lógica de control e intercambio de información del sistema se resume en la Figura C.3.

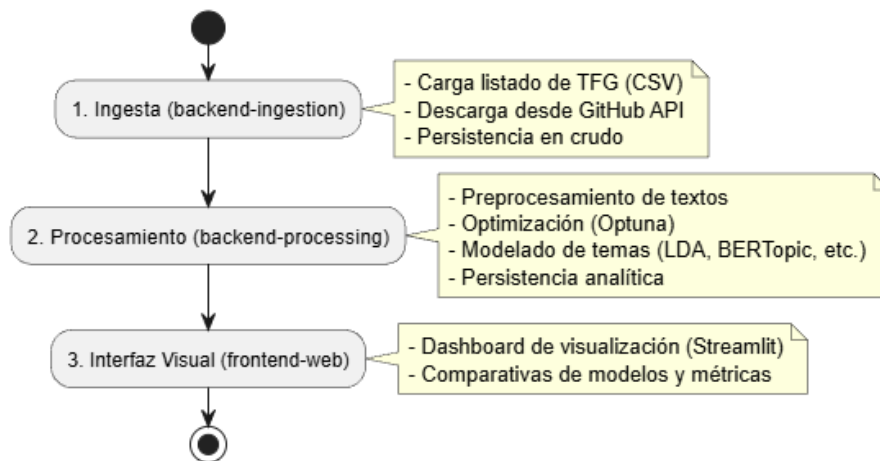


Figura C.3: Flujo procedimental general del sistema

A continuación, se desglosa el comportamiento detallado de cada una de estas tres etapas del pipeline:

Procesos de Ingesta

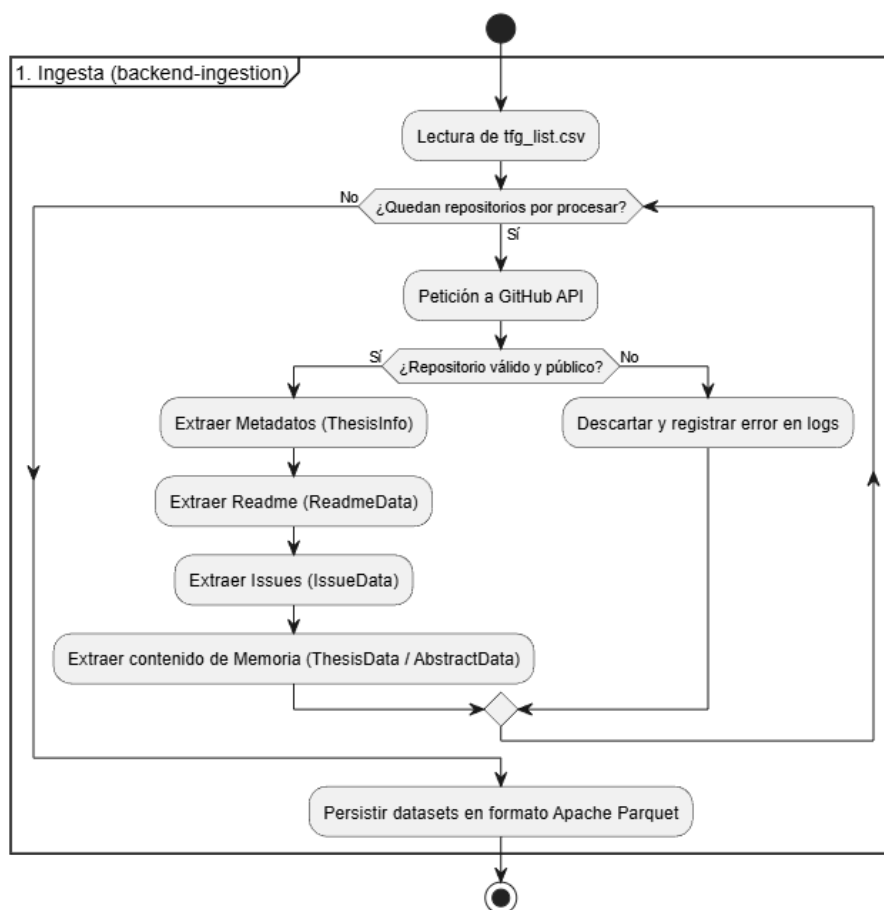


Figura C.4: Diagrama de actividad de la fase de Ingesta

Procesos de Procesamiento

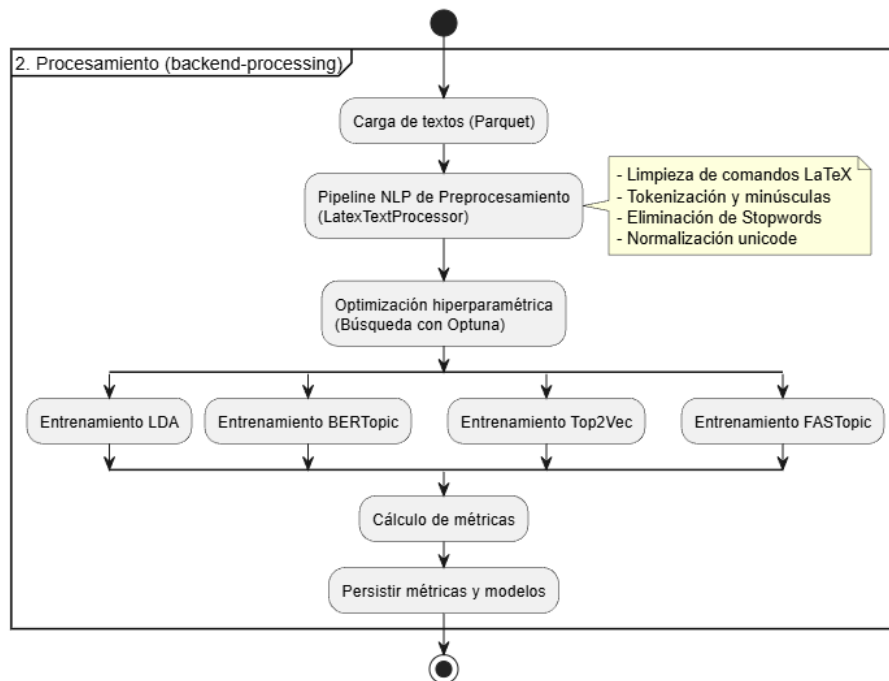


Figura C.5: Diagrama de actividad de la fase de Procesamiento

Flujo de la Interfaz de Visualización

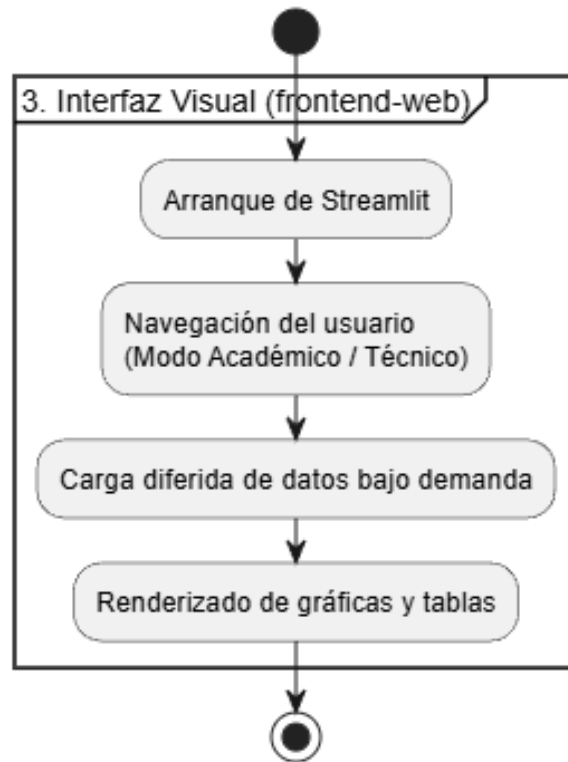


Figura C.6: Diagrama de actividad del módulo de Interfaz Visual

Apéndice D

Documentación técnica de programación

D.1. Introducción

Esta sección tiene como objetivo proporcionar una guía detallada para aquellos desarrolladores, investigadores o mantenedores que deseen comprender a nivel técnico la implementación de la aplicación *TFG Topics*, modificar su código fuente o añadir nuevas funcionalidades.

El proyecto está desarrollado en Python 3.10 [9] y se fundamenta en los principios de la Arquitectura Hexagonal (Puertos y Adaptadores) [15] para asegurar el desacoplamiento entre la lógica de negocio y las librerías externas.

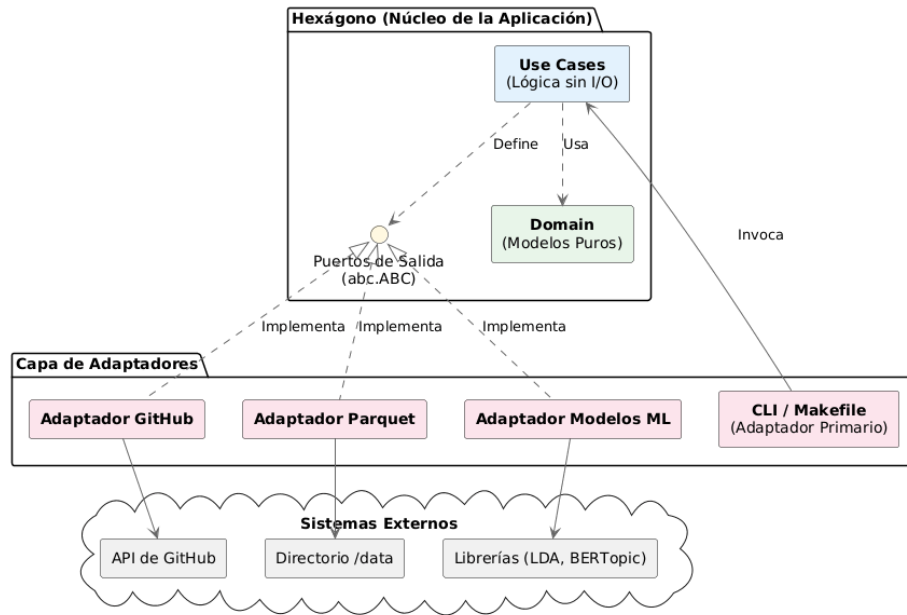


Figura D.1: Diagrama conceptual de la Arquitectura Hexagonal aplicada en los módulos backend

D.2. Arquitectura del Sistema

El sistema se divide en tres módulos claramente diferenciados:

- **backend-ingestion:** Encargado de interactuar con la **API** de GitHub, extraer los datos (*abstracts*, *readmes*, *issues*, contenido de la memoria) y almacenarlos en formato estructurado Apache Parquet.
- **backend-processing:** Motor analítico que ejecuta las fases de preprocesamiento de texto (**NLP**) y el entrenamiento de los algoritmos de modelado de temas (**LDA**, BERTopic, Top2Vec, FASTopic), evaluando hiperparámetros.
- **frontend-web:** Aplicación web ligera desarrollada en Streamlit [12], encargada únicamente de la visualización y lectura de los ficheros Parquet resultantes, sin poseer lógica de procesamiento pesada.

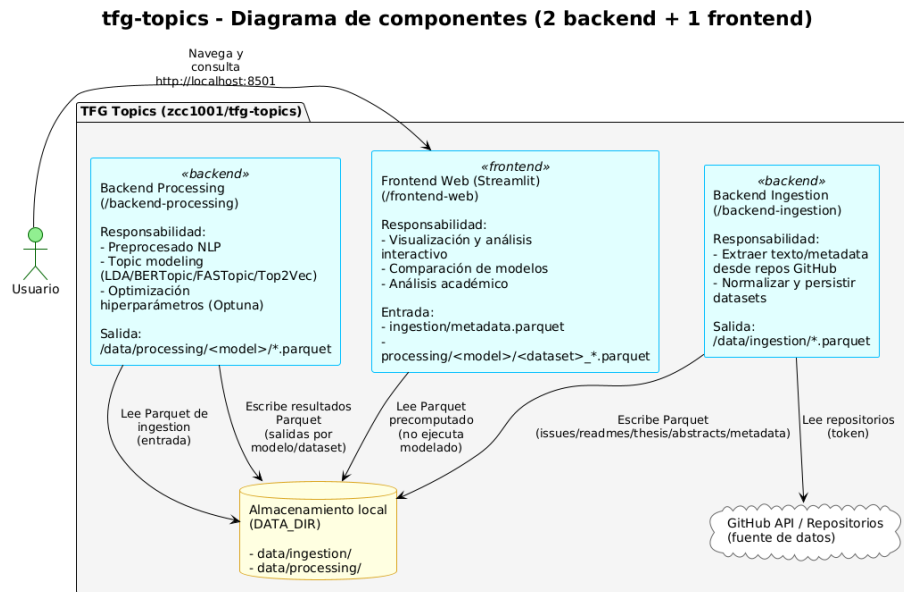


Figura D.2: Diagrama de componentes global del sistema y flujo de datos persistente

Ambos backends (*ingestion* y *processing*) implementan estrictamente una **Arquitectura Hexagonal**, estructurando su código interno en:

- *Domain*: Modelos de datos puros y lógica de negocio (independientes del framework).
- *Ports*: Interfaces abstractas (`abc.ABC`) que definen contratos de entrada y salida.
- *Use Cases*: Orquestación de la lógica de aplicación sin depender de operaciones de **I/O**.
- *Adapters*: Implementación concreta de los puertos utilizando librerías externas (ej. llamadas a la **API** de GitHub, lectura/escritura de Pandas y PyArrow, instanciación de modelos de **AI**).

D.3. Estructura de directorios

La jerarquía principal del repositorio está diseñada para separar entornos y responsabilidades:

```

tfg-topics/
|-- backend-ingestion/      # Extracción de datos de GitHub
|   |-- src/                # Código fuente
|   |-- tests/              # Pruebas unitarias
|-- backend-processing/     # Procesamiento Topic Modeling
|   |-- src/                # Código fuente
|   |-- tests/              # Pruebas del procesamiento
|-- frontend-web/           # Interfaz de usuario en Streamlit
|   |-- src/webapp/         # Páginas y componentes
|-- data/                   # Archivos resultantes
|-- docs/                   # Documentación
|-- Makefile                # Orquestador principal de tareas
|-- docker-compose.yml      # Definición de contenedores

```

D.4. Manual del programador

Estándares de Código y Calidad

Durante el desarrollo de este Trabajo de Fin de Grado, se ha prestado especial atención a la calidad, legibilidad y mantenibilidad del software. Para garantizar que el proyecto pueda ser comprendido o ampliado por futuros estudiantes e investigadores sin problemas de comprensión, se han integrado herramientas automáticas que unifican el estilo de programación, logrando que todo el código base mantenga una estructura coherente y profesional.

Las herramientas configuradas para este propósito son:

- **Formateo automático con Black e isort:** *Black* [1] es la herramienta elegida para dar formato al código de forma determinista (gestionando la alineación, espacios y saltos de línea), limitando la longitud máxima a 88 caracteres por línea. Adicionalmente, *isort* [5] se encarga de agrupar y ordenar alfabéticamente las importaciones al principio de cada archivo. La adopción de estas herramientas exige al desarrollador de realizar el formateo de forma manual.
- **Tipado estricto con Mypy:** A pesar de que Python es un lenguaje de tipado dinámico por naturaleza, en la arquitectura de este proyecto se ha exigido el uso de anotaciones de tipo (*type hints*) para todas las variables, parámetros y retornos de funciones. *Mypy* [6] actúa como un analizador estático que valida el código antes de su ejecución,

previniendo así un gran volumen de errores en tiempo de ejecución derivados de la incompatibilidad de tipos de datos.

- **Revisiones automáticas (Pre-commit):** Con el fin de evitar que se introduzca código en el repositorio que incumpla las reglas de estilo o tipado, se ha implementado un sistema de ganchos de pre-confirmación (*pre-commit hooks*) [7]. Esta herramienta actúa como un filtro de calidad: cada vez que se intenta realizar un *commit* en Git, se analizan los cambios automáticamente. Si se detectan infracciones de formato, el sistema las corrige en la medida de lo posible; si existen errores lógicos o de tipado, el proceso se aborta hasta que sean solventados.

Gestión de Errores y Trazabilidad

Para facilitar la depuración y asegurar un control riguroso sobre el flujo de ejecución (especialmente en procesos largos y desatendidos como la extracción o el entrenamiento de modelos), se han definido dos directrices principales en el desarrollo:

- **Manejo explícito de excepciones:** Se ha prohibido la práctica de capturar y silenciar errores de forma genérica (por ejemplo, mediante bloques `except Exception: pass`). En su lugar, cuando se produce una anomalía (como un fallo de red o la ausencia de un fichero), el sistema debe interceptarla y lanzar una excepción personalizada propia del dominio del proyecto. Esto garantiza que problemas como un fallo en la **API** de GitHub no deriven de forma silenciosa en errores incomprensibles durante el procesamiento de los datos.
- **Trazabilidad mediante Logging:** Se ha descartado el uso de la instrucción básica `print()` para informar del estado de la ejecución. En su lugar, se ha integrado el sistema de **logging** nativo de Python, el cual permite categorizar los mensajes según su nivel de severidad: **INFO** (para trazar el progreso normal de los algoritmos), **WARNING** (para advertencias no bloqueantes) y **ERROR** (para registrar caídas críticas). Esta aproximación ha resultado fundamental para volcar el historial de ejecución a ficheros de texto y poder investigar las incidencias de forma asíncrona.

D.5. Compilación, instalación y ejecución del proyecto

Con el objetivo de simplificar el despliegue y la reproducibilidad de los experimentos, el proyecto abstrae la complejidad del entorno mediante el uso de contenedores Docker [14] y el orquestador **Make**. El fichero **Makefile** principal ha sido diseñado para proporcionar un sistema de ejecución polivalente, controlado por la variable **MODE**, que admite tres flujos de trabajo distintos:

- **MODE=release** (Por defecto): Descarga y utiliza imágenes precompiladas alojadas en *GitHub Container Registry* (**GHCR**). Es la vía más rápida y recomendada para la ejecución sin alteración del código fuente.
- **MODE=docker**: Compila las imágenes Docker localmente a partir del código fuente. Ideal para probar cambios en la infraestructura o en las dependencias.
- **MODE=local**: Ejecuta el código de forma nativa utilizando el intérprete de Python del sistema anfitrión, sin aislar el entorno en contenedores.

Variables de entorno y configuración

La configuración operativa del sistema se controla mediante variables de entorno. Esta estrategia permite mantener el mismo código para entornos locales, contenedores y ejecución en la nube, reduciendo acoplamientos y facilitando la reproducibilidad de los experimentos.

- **DATA_DIR**: Directorio base de datos del proyecto. Es consumido por *backend-ingestion*, *backend-processing* y *frontend-web*. Si no se define, se utiliza por defecto la carpeta **data/** en la raíz del repositorio.
- **GITHUB_TOKEN**: Token personal de GitHub para autenticación en la **API** REST. Es opcional, pero muy recomendable para evitar bloqueos por límites de peticiones.
- **REPOS_CSV_FILE_NAME**: Nombre del fichero CSV con la lista de repositorios a ingerir (por defecto **tfg_list.csv**).
- **LOG_DIR**: Ruta personalizada para persistir logs. Si no se define, se utiliza **<DATA_DIR>/logs**.

Contrato de datos Parquet entre módulos

El sistema está diseñado como una cadena de procesamiento por etapas, con contratos de intercambio explícitos en Apache Parquet. Este enfoque desacopla los módulos y permite relanzar componentes sin rehacer todo el pipeline.

Salida mínima esperada de ingesta (`data/ingestion`):

- `issues.parquet`
- `readmes.parquet`
- `thesis.parquet`
- `abstracts.parquet`
- `metadata.parquet`

Salida mínima esperada de procesamiento (`data/processing/<modelo>`):

- `<dataset>_model_summary.parquet`
- `<dataset>_metrics.parquet`
- `<dataset>_document_topics.parquet`
- `<dataset>_topics.parquet`
- `<dataset>_params.parquet`
- `<dataset>_topic_coordinates.parquet`
- `<dataset>_best_hyperparameters.parquet`
- `<dataset>_hyperparameter_trials.parquet`

La interfaz web consume estos artefactos en modo lectura, por lo que cualquier ausencia o desalineación en nombres o columnas impacta directamente en la visualización.

Trazabilidad y observabilidad

Los tres módulos principales incluyen trazabilidad homogénea mediante `logging` con salida dual a consola y fichero rotatorio. Esta decisión facilita tanto la depuración interactiva como la auditoría de ejecuciones desatendidas.

- Carpeta de logs por defecto: `<DATA_DIR>/logs`
- Ficheros generados: `ingestion.log`, `processing.log`, `frontend.log`
- Política de rotación: 10 MB por fichero con 5 copias de respaldo

Comandos principales de orquestación

Para facilitar el uso del sistema y evitar la ejecución manual prolongada, se han definido alias en el `Makefile` que abstraen las invocaciones a Docker Compose [14] y a los distintos scripts de Python:

- `make start`: Levanta la interfaz gráfica (*frontend-web*).
- `make ingest-all`: Ejecuta la extracción de todos los conjuntos de datos contemplados (*issues*, *readmes*, *thesis*, *abstracts*) desde GitHub.
- `make process-all`: Ejecuta secuencialmente el entrenamiento y la evaluación de los cuatro modelos de **AI** sobre todos los conjuntos de datos.
- `make pipeline`: Orquesta el flujo completo de forma desatendida, encadenando `ingest-all`, `process-all` y `start`.
- `make pull`: Descarga explícitamente las últimas imágenes publicadas.
- `make docker-build`: Fuerza la reconstrucción de las imágenes Docker locales (comando complementario para `MODE=docker`).

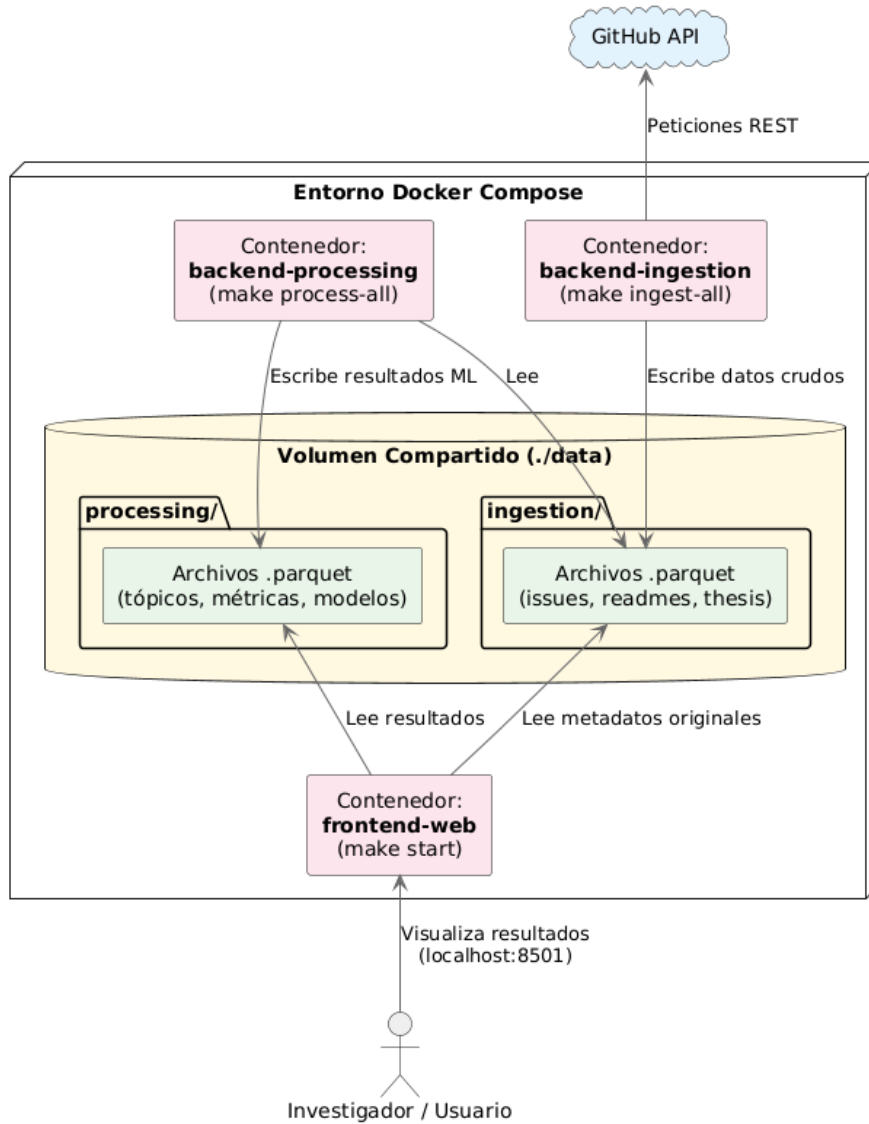


Figura D.3: Diagrama de interacción y flujo de datos en Apache Parquet entre contenedores Docker

Configuración de Credenciales

Para la fase de extracción de datos (**backend-ingestion**), el sistema realiza múltiples llamadas a la **API REST** de GitHub [4]. Para evitar bloqueos derivados de las políticas de límite de peticiones (*rate limits*) impuestas a los clientes anónimos [11], resulta fundamental configurar un token de acceso personal.

El orquestador está diseñado para inyectar automáticamente la variable de entorno `GITHUB_TOKEN` en los contenedores o rutinas locales. Antes de lanzar cualquier proceso de ingesta, el investigador debe exportar su clave en la terminal activa:

En entornos de base Unix (Linux / macOS):

```
export GITHUB_TOKEN="tu_token_generado_aqui"
```

En entornos Windows (PowerShell):

```
$env:GITHUB_TOKEN="tu_token_generado_aqui"
```

Entorno Local de Desarrollo

Para los escenarios donde el investigador requiere modificar el código fuente localmente y depurar los cambios de forma interactiva, se ha previsto la variante `MODE=local`:

1. El equipo anfitrión debe contar con Python 3.10 y Conda (o equivalente).
2. Se requiere instalar las dependencias de cada módulo (ej. mediante `pip install -r requirements.txt`).
3. Las reglas de extracción o procesamiento se invocan forzando explícitamente el modo local. Por ejemplo, para lanzar el flujo de ingesta nativamente:

```
make ingestion INGEST="all" MODE=local
```

4. Del mismo modo, se ejecutan las capas homólogas: `make processing MODE=local` o `make frontend MODE=local`.

Integración con GitHub Codespaces

Como parte de los requisitos de accesibilidad universal propuestos para el **TFG**, se ha configurado un entorno de desarrollo basado en la nube a través de la herramienta **GitHub Codespaces** [3]. El repositorio incluye la definición del entorno (`devcontainer.json`), que se encarga de automatizar la instalación de Python y todas las dependencias subyacentes.

Al instanciar un Codespace directamente desde el navegador web, se proporciona al usuario una máquina virtual preconfigurada. Dentro de este entorno aislado, las invocaciones deben realizarse forzando el modo local, ya que las dependencias se han volcado sobre la propia máquina virtual del contenedor de Codespaces:

```
make ingest-all MODE=local
make process-all MODE=local
make start MODE=local
```

Guía de calidad y validación continua

La base del proyecto se valida mediante tres capas complementarias:

- **Formateo y estilo:** `black`, `isort`, `flake8`
- **Verificación de tipos:** `mypy`
- **Pre-commit:** para impedir commits con deuda técnica evitable

Flujo recomendado antes de integrar cambios:

```
pre-commit run --all-files
```

D.6. Pruebas del sistema

La verificación funcional se delega en el framework **Pytest** [8]. Gracias a la arquitectura hexagonal, la inmensa mayoría del código se prueba mediante test unitarios de ejecución rápida, aislando las dependencias pesadas o de **I/O** mediante dobles de prueba (*mocks*).

Cada submódulo lógico del backend dispone de su propio directorio `tests/`. Para ejecutar la batería completa de pruebas unitarias y comprobar su validez, el desarrollador debe situarse en el directorio del módulo correspondiente e invocar el marco de pruebas:

```
cd backend-ingestion
pytest tests/
```

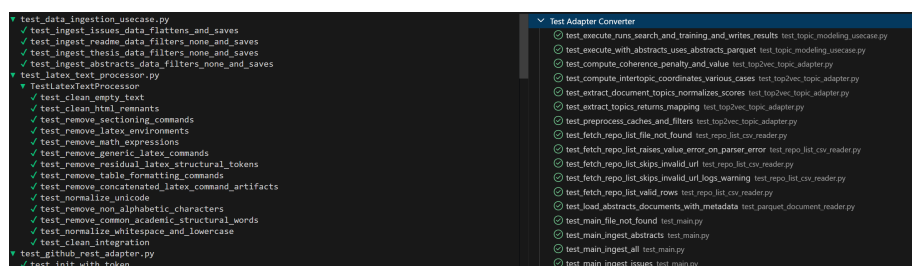


Figura D.4: Ejemplo de ejecución del pipeline de pruebas con Pytest

Resolución de incidencias frecuentes

- **Error de CSV no encontrado en ingesta:** Verificar `DATA_DIR` y `REPOS_CSV_FILE_NAME`. El sistema espera encontrar el fichero en `<DATA_DIR>/<REPOS_CSV_FILE_NAME>`.
- **Límites de GitHub alcanzados:** Definir `GITHUB_TOKEN` antes de la ingesta para aumentar cuota de peticiones.
- **Frontend sin datos:** Confirmar que existen ficheros Parquet de procesamiento para el modelo y dataset seleccionados.
- **Modelo no soportado:** Revisar que el valor de `-model` pertenezca al conjunto implementado (`lda`, `bertopic`, `top2vec`, `fastopic`).
- **Resultados incoherentes tras cambiar datos:** Relanzar `make process-all` para forzar regeneración completa de resultados.

Apéndice *E*

Documentación de usuario

E.1. Introducción

La presente sección constituye el manual de usuario de la aplicación *TFG Topics*, desarrollada como parte del Trabajo de Fin de Grado del grado de Ingeniería Informática de la Universidad de Burgos. El sistema consiste en una aplicación web diseñada para explorar y comparar los resultados obtenidos del análisis temático sobre memorias y repositorios de Trabajos Fin de Grado disponibles públicamente en GitHub.

El proyecto se estructura en tres módulos principales de forma transparente para el usuario final:

- **Ingesta (*backend-ingestion*):** Módulo encargado de la extracción de datos desde la **API** de GitHub y su almacenamiento estructurado en ficheros formato Parquet.
- **Procesamiento (*backend-processing*):** Módulo analítico que procesa los textos extraídos, entrena los modelos de modelado de temas (**LDA**, BERTopic, Top2Vec, FASTopic) y evalúa los hiperparámetros óptimos.
- **Interfaz Web (*frontend-web*):** Aplicación interactiva desarrollada en Streamlit [12] que permite la visualización y exploración de los resultados almacenados.

E.2. Requisitos del sistema

Para asegurar el correcto funcionamiento de la plataforma, el equipo del usuario debe cumplir con una serie de requisitos dependiendo del modo de uso elegido:

- **Modo Visualización:** Únicamente se requiere disponer de un navegador web moderno (Google Chrome, Mozilla Firefox, Edge) para acceder a una instancia de la aplicación ya desplegada.
- **Modo Recomendado (Docker):** Es necesario tener instalado *Docker Engine* y *Docker Compose* (o *Docker Desktop* en entornos Windows/macOS). Esta es la forma más sencilla de ejecutar el proyecto, ya que utiliza imágenes precompiladas y no requiere configurar el entorno de programación de Python localmente.
- **Modo Desarrollo Local:** Requiere tener instalado Python 3.10, gestor de dependencias (*pip* o *Conda*), la herramienta *make* y el sistema de control de versiones *Git*.

E.3. Instalación y despliegue

El sistema se ha empaquetado utilizando contenedores Docker [14] alojados en **GITHUB CONTAINER REGISTRY (GHCR)**, lo que permite un despliegue rápido sin necesidad de compilar el código fuente.

Para ejecutar la aplicación completa en un equipo local mediante Docker, se deben seguir los siguientes pasos:

1. **Clonar el repositorio:** Obtener el código del proyecto desde el repositorio oficial mediante la línea de comandos:

```
git clone https://github.com/zcc1001/tfg-topics
cd tfg-topics
```

2. **Actualizar las imágenes:** Descargar las últimas versiones de los contenedores utilizando la herramienta *Make*:

```
make pull
```

3. **Ejecutar el flujo de trabajo:** El repositorio incluye comandos automatizados. Para ejecutar la extracción de datos, el procesamiento de los modelos y lanzar la interfaz web de manera secuencial y desatendida, se debe ejecutar:

```
make pipeline
```

4. **Arrancar la interfaz aislada:** Si los datos ya han sido procesados previamente y se desea arrancar únicamente la interfaz visual, se utiliza el comando:

```
make start
```

Una vez finalizado el proceso de despliegue, la interfaz web estará disponible accediendo a la **URL** `http://localhost:8501` desde cualquier navegador web en la máquina anfitriona.

E.4. Manual del usuario de la interfaz

La aplicación web ofrece una interfaz gráfica intuitiva con un menú de navegación situado en el panel lateral izquierdo. Este menú permite alternar entre dos enfoques principales de análisis: el **Modo Académico** y el **Modo Técnico** como se muestra en la Figura E.1.

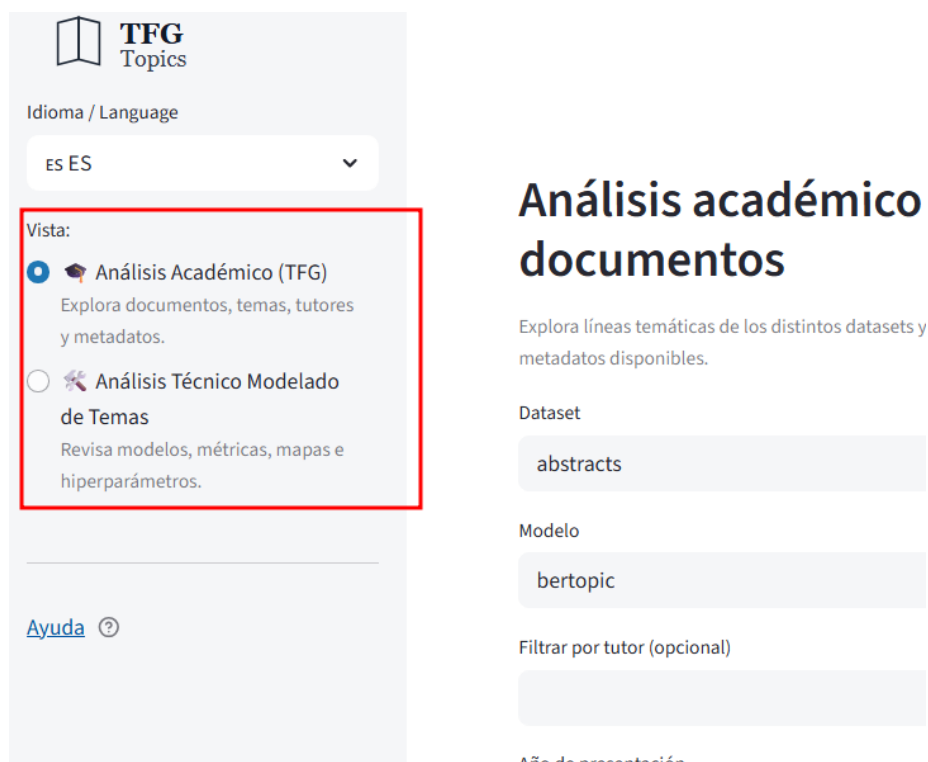


Figura E.1: Menu lateral de navegación

Modo Académico

Este modo está orientado al análisis cualitativo y exploratorio de los proyectos, y está pensado para docentes, miembros de tribunal o investigadores que deseen comprender el contenido de los trabajos sin entrar en detalles algorítmicos complejos.

Como se aprecia en la Figura E.2, la interfaz proporciona:

- **Filtros interactivos:** Permite afinar la búsqueda de proyectos utilizando controles interactivos. El usuario puede buscar por nombre de tutor (campo de texto opcional), seleccionar un rango de años de presentación y acotar por nota mediante un control deslizante de rango.
- **Exploración de proyectos:** Una vez aplicados los filtros, se muestra el listado de resultados. Cada trabajo se presenta en una tarjeta detallada que incluye el título, un resumen descriptivo y una ficha con la *Información Académica* (Autor, Tutor, Centro, Curso, etc.).

- **Distribución de temáticas:** Muestra gráficos visuales y resúmenes sobre cuáles son las tecnologías, metodologías y temáticas más frecuentemente tratadas por el alumnado en los diferentes cursos académicos (ver Figura E.3).

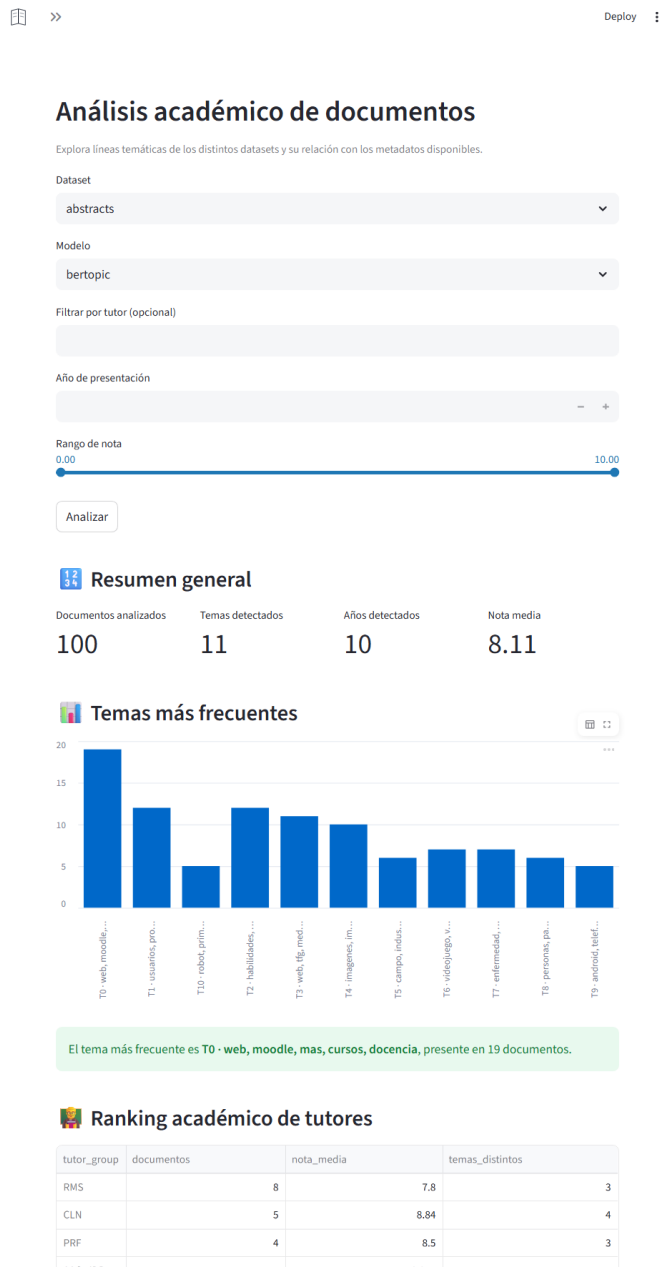


Figura E.2: Vista Análisis Académico

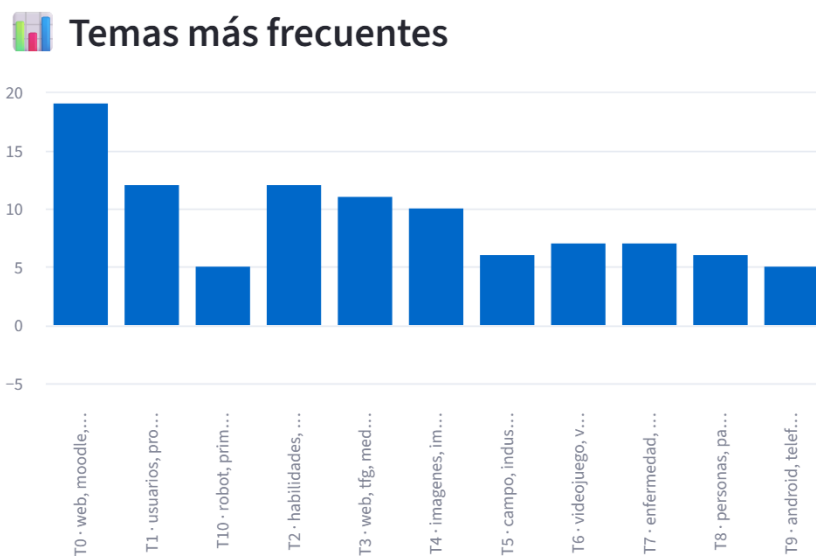


Figura E.3: Vista Distribución Académica

Modo Técnico

Orientado al análisis de los resultados y métricas obtenidas por los distintos algoritmos de Procesamiento de Lenguaje Natural (**NLP**). Al margen de los controles globales situados en la parte inferior del menú lateral (como el selector de **Idioma / Language** y el selector de **Vista**), una vez activado el *Análisis Técnico*, el menú se despliega ofreciendo una estructura jerárquica dividida en tres bloques principales:

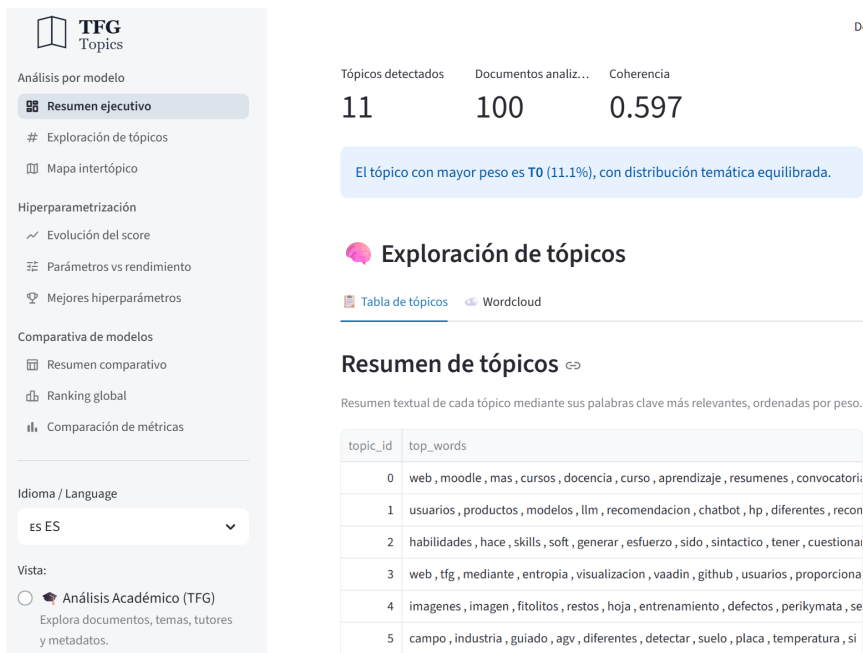


Figura E.4: Vista Modo Técnico

Análisis por modelo

En esta sección el usuario puede explorar los resultados detallados de un algoritmo y conjunto de datos específicos. Incluye las siguientes opciones de navegación:

- **Resumen ejecutivo:** (Figura E.5) Encabeza este apartado ofreciendo métricas de alto nivel, tales como el número de temas detectados, la cantidad de documentos analizados y el valor de coherencia global del modelo. Además, destaca visualmente el tema con mayor peso o distribución.
- **Exploración de temas:** Proporciona una tabla interactiva (*Tabla de temas*) que relaciona el identificador de cada tema con sus palabras clave más relevantes, y ofrece vistas alternativas como nubes de palabras (*Wordcloud*, Figura E.6).
- **Mapa intertópico:** Representación bidimensional para analizar la distancia semántica y relación entre los distintos temas descubiertos (Figura E.7).

Análisis por modelo

Selecciona el conjunto de documentos y el modelo de tópicos para analizar sus resultados precomputados.

Selecciona un dataset

abstracts



Selecciona un modelo

bertopic



Resumen ejecutivo

Tópicos detectados

11

Documentos analiz...

100

Coherencia

0.597

El tópico con mayor peso es T0 (11.1%), con distribución temática equilibrada.

Figura E.5: Vista Resumen Ejecutivo - Análisis por modelo



Mapa de distancia entre tópicos

Proyección bidimensional de los tópicos en el espacio latente. La distancia entre círculos refleja su separación semántica.

Los tamaños de los círculos indican la relevancia relativa de cada tópico.

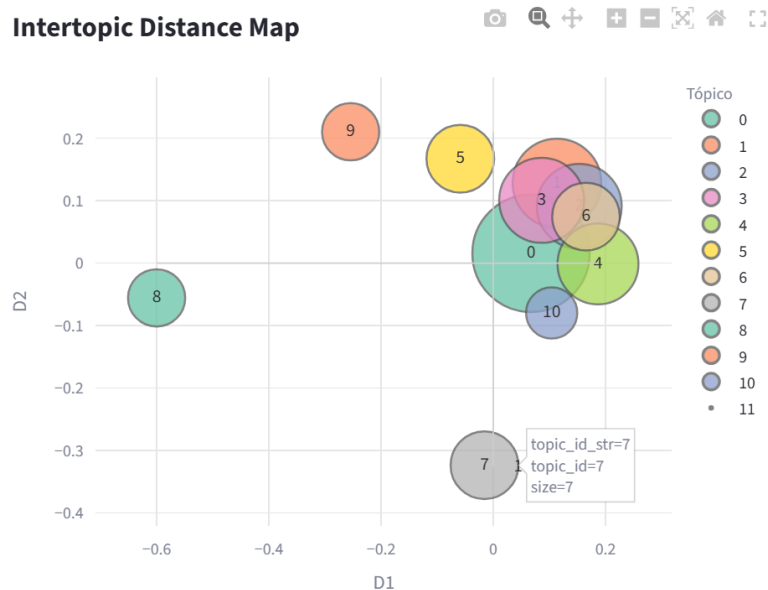


Figura E.7: Vista Mapa Intertópico

Hiperparametrización

Permite observar la evolución y los resultados del proceso de ajuste automático de parámetros de los modelos, a través de las siguientes subsecciones:

- **Evolución del score:** Muestra de forma general cómo ha mejorado la métrica objetivo a lo largo del proceso de optimización.
- **Parámetros vs rendimiento:** (Figura E.8) Presenta una gráfica interactiva de dispersión (*scatter plot*) que muestra el valor de coherencia obtenido en cada intento (*trial*). Facilita la identificación del mejor modelo, marcado de forma destacada (por ejemplo, con una estrella roja indicando el *Best trial*).
- **Mejores hiperparámetros:** (Figura E.9) Muestra la configuración exacta que maximizó el rendimiento del modelo durante el entrenamiento.



Evolución del score

Evolución de la búsqueda de hiperparámetros

Evolución del valor de coherencia a lo largo de las distintas pruebas realizadas durante la optimización.

Score (coherence) por trial

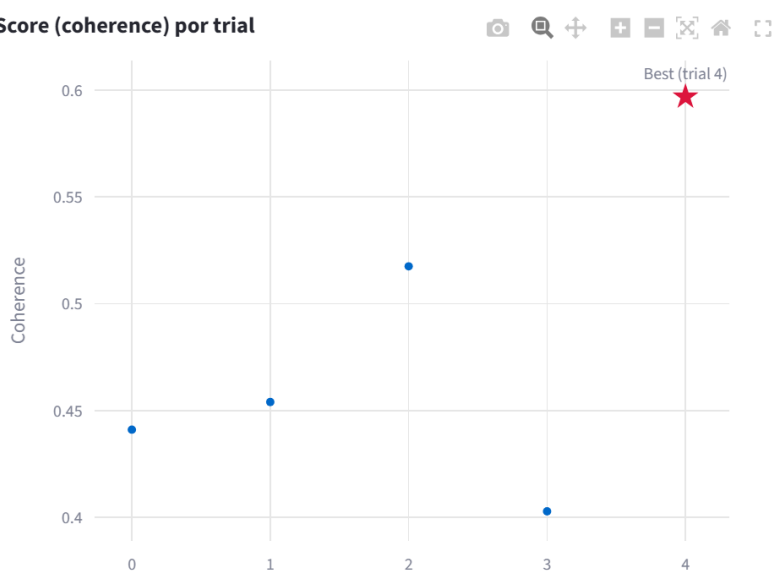


Figura E.8: Vista Evolución del score - Hiperparametrización

Mejor configuración encontrada

Configuración del modelo que maximiza la coherencia según el proceso de optimización.

Esta configuración se utiliza como resultado final del modelo para el análisis posterior.

Modelo	Best score (coheren...	Num. tópicos
bertopic	0.597	0

Parámetros seleccionados:

```
▼ {  
  "dataset" : "abstracts"  
  "min_topic_size" : "4"  
  "n_neighbors" : "9"  
  "n_components" : "23"  
  "ngram_min" : "1"  
  "ngram_max" : "1"  
}
```

Figura E.9: Vista Mejores hiperparámetros - Hiperparametrización

Comparativa de modelos

Aglutina las visiones que enfrentan los resultados de los cuatro modelos empleados (LDA [17], BERTopic [10], Top2Vec [16] y FASTopic [2]) bajo diferentes métricas cuantitativas. Comprende las vistas de:

- **Resumen comparativo y Ranking global:** Evalúan el nivel de diversidad temática y coherencia de manera conjunta para establecer qué algoritmo se comporta mejor (Figura E.10).
- **Comparación de métricas:** Gráficos cruzados para contrastar los distintos aspectos de evaluación entre los algoritmos entrenados (Figura E.11).

Ranking global

Ranking relativo entre los modelos seleccionados para el dataset actual.

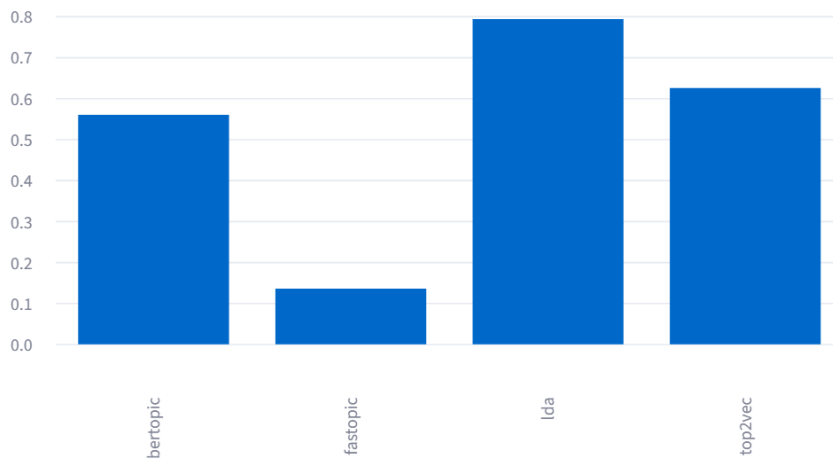


Figura E.10: Vista Comparativa Modelo - Ranking modelos

Comparativa de modelos

Selecciona un dataset

abstracts

Selecciona modelos a comparar

bertopic x

fastopic x

lda x

top2vec x

Resumen comparativo

	dataset	model_name	run_id	num_topics	coherence	runtime_seconds	final_score
2	abstracts	lda	lda_hyperparameter_search_20260504T210413	6	0.4905	0.7334	0.7928
3	abstracts	top2vec	top2vec_hyperparameter_search_20260504T213704	17	0.5525	14.0576	0.625
0	abstracts	bertopic	bertopic_hyperparameter_search_20260504T212026	11	0.4702	6.618	0.5594
1	abstracts	fastopic	fastopic_hyperparameter_search_20260504T215944	6	0.3655	9.2257	0.136

El score final combina coherencia y tiempo de ejecución, normalizados entre modelos disponibles.

Modelo recomendado: lda (score = 0.79)

Figura E.11: Vista Comparativa Modelos - Resumen comparativo

Anexo de sostenibilización curricular

F.1. Introducción

El presente anexo recoge una reflexión crítica sobre la alineación del TFG con los principios de sostenibilidad y responsabilidad social, en concordancia con las directrices establecidas por la CONFERENCIA DE RECTORES DE LAS UNIVERSIDADES ESPAÑOLAS (CRUE).

La ingeniería del software, tradicionalmente percibida como una disciplina aséptica, tiene un impacto profundo y directo en el consumo de recursos naturales, el desarrollo socioeconómico y la equidad social. Por tanto, el diseño, arquitectura y desarrollo de este sistema de análisis temático se ha llevado a cabo bajo el prisma de sostenibilidad ambiental, social y económica.

A continuación, en la Tabla F.1, se resumen las principales decisiones adoptadas y su impacto directo en la sostenibilidad.

F.2. Sostenibilidad Ambiental

El auge reciente de la Inteligencia Artificial y de los modelos de Procesamiento de Lenguaje Natural (NLP) ha traído consigo un incremento alarmante en la huella de carbono, debido a la ingente cantidad de recursos computacionales y energéticos necesarios para su entrenamiento. En este proyecto, se han aplicado principios de *Green AI* (IA verde) para minimizar drásticamente el impacto medioambiental.

Decisión del proyecto	Dimensión Sostenible
Uso de IA ligera local y Parquet	Ambiental
Desarrollo de herramienta analítica	Social
Anonimización de datos (RGPD)	Social / Ética
Licenciamiento de código abierto	Económica

Tabla F.1: Resumen del impacto de las decisiones técnicas en la sostenibilidad.

En primer lugar, se ha evitado la dependencia de modelos de lenguaje cerrados en la nube que requieren centros de datos masivos. En su lugar, se ha optado por modelos probabilísticos ligeros (como **LDA**) y modelos basados en Transformers de tamaño reducido (como BERTopic y Top2Vec) que pueden ser ejecutados y afinados localmente utilizando un hardware doméstico estándar.

Además, la arquitectura de persistencia de datos ha sido diseñada para ser altamente eficiente. La transición de formatos tradicionales en texto plano (como JSON o **CSV**) hacia el uso exclusivo de Apache Parquet ha permitido comprimir drásticamente el espacio de almacenamiento. Parquet, al ser un formato columnar optimizado, minimiza las operaciones de lectura y escritura en disco (I/O), reduciendo notablemente el consumo energético del procesador y el disco durante las pesadas fases de tratamiento de texto.

F.3. Sostenibilidad Social y Ética

Desde el punto de vista del impacto humano, el proyecto contribuye de forma directa al Educación de Calidad, Industria, Innovación e Infraestructura. La herramienta desarrollada tiene un fin puramente académico y divulgativo: proporcionar a investigadores, docentes y futuros estudiantes una plataforma analítica intuitiva para comprender las tendencias, tecnologías y temáticas que se están investigando en el ámbito universitario. Al extraer el conocimiento latente de memorias de **TFG**, se facilita la transferencia de conocimiento, se inspiran nuevas líneas de investigación para el alumnado y se evita la redundancia académica.

F.4. Sostenibilidad Económica

El ecosistema de este desarrollo se ha sustentado de forma exclusiva sobre tecnologías libres y gratuitas. El *backend* (Python, Pandas, Optuna) y el motor de visualización (Streamlit) no exigen licencias comerciales de pago.

Asimismo, la ligereza de la interfaz web permite que el producto final pueda ser albergado en plataformas en la nube de capa gratuita (como *GitHub Container Registry* o *Streamlit Community Cloud*), garantizando una disponibilidad analítica universal a coste cero de mantenimiento operativo.

F.5. Conclusión

En conclusión, la elaboración de este TFG ha supuesto una oportunidad invaluable para interiorizar y poner en práctica las competencias transversales de sostenibilidad impulsadas por la CRUE. Las decisiones de diseño técnico adoptadas desde la selección del formato Parquet para reducir el consumo energético, hasta la adopción de una licencia de código abierto para democratizar su uso y la anonimización proactiva de los datos recogidos demuestran que es posible concebir productos de software que sean técnicamente robustos a la par que social, económica y medioambientalmente responsables. Este proyecto contribuye a través de una praxis ética y respetuosa con su entorno material y humano.

Acrónimos

AI Artificial Intelligence. 35, 40

API Application Programming Interface. 4, 7, 11, 12, 15, 23, 26, 34, 35, 37, 38, 41, 45

CRUE Conferencia de Rectores de las Universidades Españolas. 59, 61

CSV Comma-Separated Values. 22, 23, 60

CU Caso de Uso. i, iii, v, 9, 12, 13, 14, 15, 17, 19, 20, 26

GHCR GitHub Container Registry. 38, 46

I/O Input/Output. 4, 35, 43

JSON JavaScript Object Notation. 22, 23

LDA Latent Dirichlet Allocation. 4, 10, 24, 26, 28, 34, 45, 57, 60

NLP Natural Language Processing. 1, 4, 10, 13, 16, 26, 34, 50, 59

RF Requisito Funcional. v, 9, 11, 19, 20

RNF Requisito No Funcional. 9, 12

TFG Trabajo de Fin de Grado. 1, 5, 6, 7, 10, 13, 15, 23, 27, 42, 59, 60, 61

URL Uniform Resource Locator. 47

Bibliografía

- [1] Black documentation. <https://black.readthedocs.io/en/stable/>.
- [2] Fastopic: FASTopic. <https://github.com/bobxwu/fastopic>.
- [3] Github codespaces documentation. <https://docs.github.com/en/codespaces>.
- [4] Github rest api documentation. <https://docs.github.com/en/rest>.
- [5] isort documentation. <https://pycqa.github.io/isort/>.
- [6] mypy documentation. <https://mypy.readthedocs.io/en/stable/>.
- [7] pre-commit documentation. <https://pre-commit.com/>.
- [8] pytest documentation. <https://docs.pytest.org/en/stable/>.
- [9] Python 3.10.19 Documentation. <https://docs.python.org/3.10/>.
- [10] Quick Start - BERTopic. https://maartengr.github.io/BERTopic/getting_started/quickstart/quickstart.html.
- [11] Rate limits for the rest api. <https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api>.
- [12] Streamlit Docs. <https://docs.streamlit.io/>.
- [13] User Guide — pandas documentation. https://pandas.pydata.org/docs/user_guide/index.html.
- [14] Docker manuals. <https://docs.docker.com/manuals/>, 09:11:23 +0100 +0100.

- [15] Hexagonal Architecture and Clean Architecture (with examples). <https://dev.to/dyarleniber/hexagonal-architecture-and-clean-architecture-with-examples-48oi>, 2022.
- [16] Dimo Angelov. Top2Vec: Distributed Representations of Topics, 2020. <http://arxiv.org/abs/2008.09470>.
- [17] David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.